

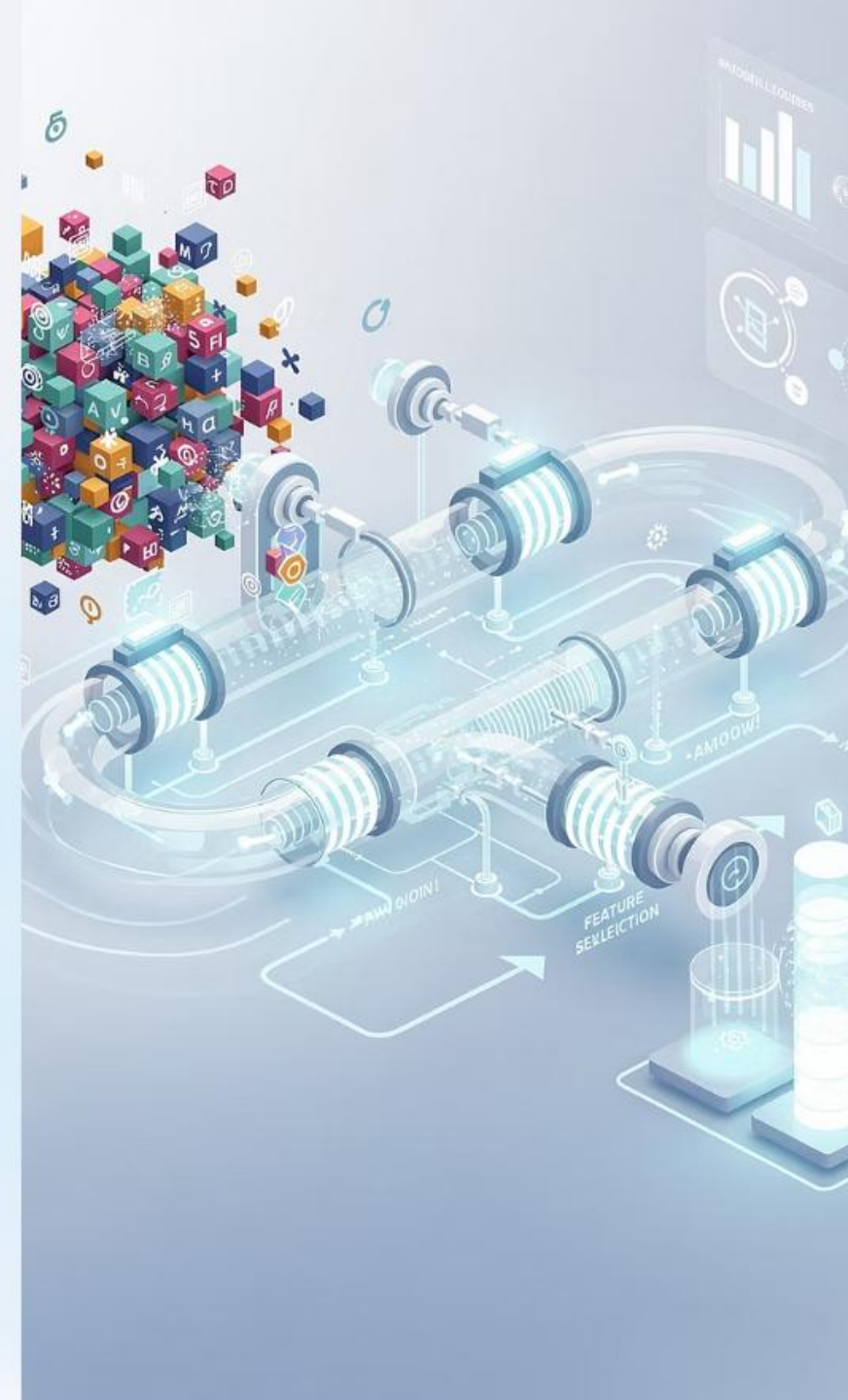
Erasmus Mundus Joint Master Artificial Intelligence for
Sustainable Societies (AISS)

Topics on Machine Learning and its Applications

Feature engineering

Pedro Strecht

1st part



Feature engineering (1st part)

Introduction

Feature creation

Feature transformation

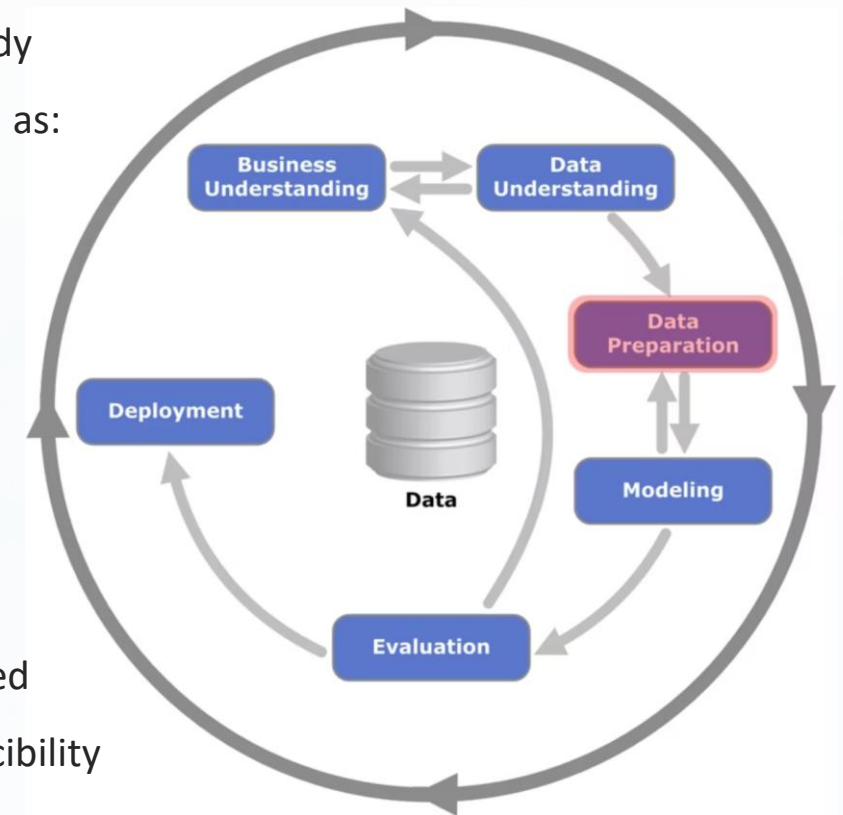
Case studies of feature engineering

Feature engineering

Introduction

In CRISP-DM **Feature engineering** corresponds to **Data preparation**

- The **data preparation** phase transforms raw data into a refined dataset ready for analysis. This involves a series of steps tailored to the specific data, such as:
 1. **Consolidating** data from various sources;
 2. **Standardizing** column names for consistency;
 3. **Cleaning** data by addressing errors and inconsistencies;
 4. **Identifying and handling** outliers;
 5. **Transforming** data into a suitable format for analysis tools;
 6. **Partitioning** data into training, testing, and validation sets.
- Once finalized, these cleaning and transformation steps are often automated within an **ETL** (Extract, Transform, Load) process for efficiency and reproducibility



Feature engineering

Introduction

■ Motivation

- Why models fail even with good learning algorithms?
- "Garbage in, garbage out"
- Model performance can also often be limited by the **features**, not by the learning algorithm



Feature engineering

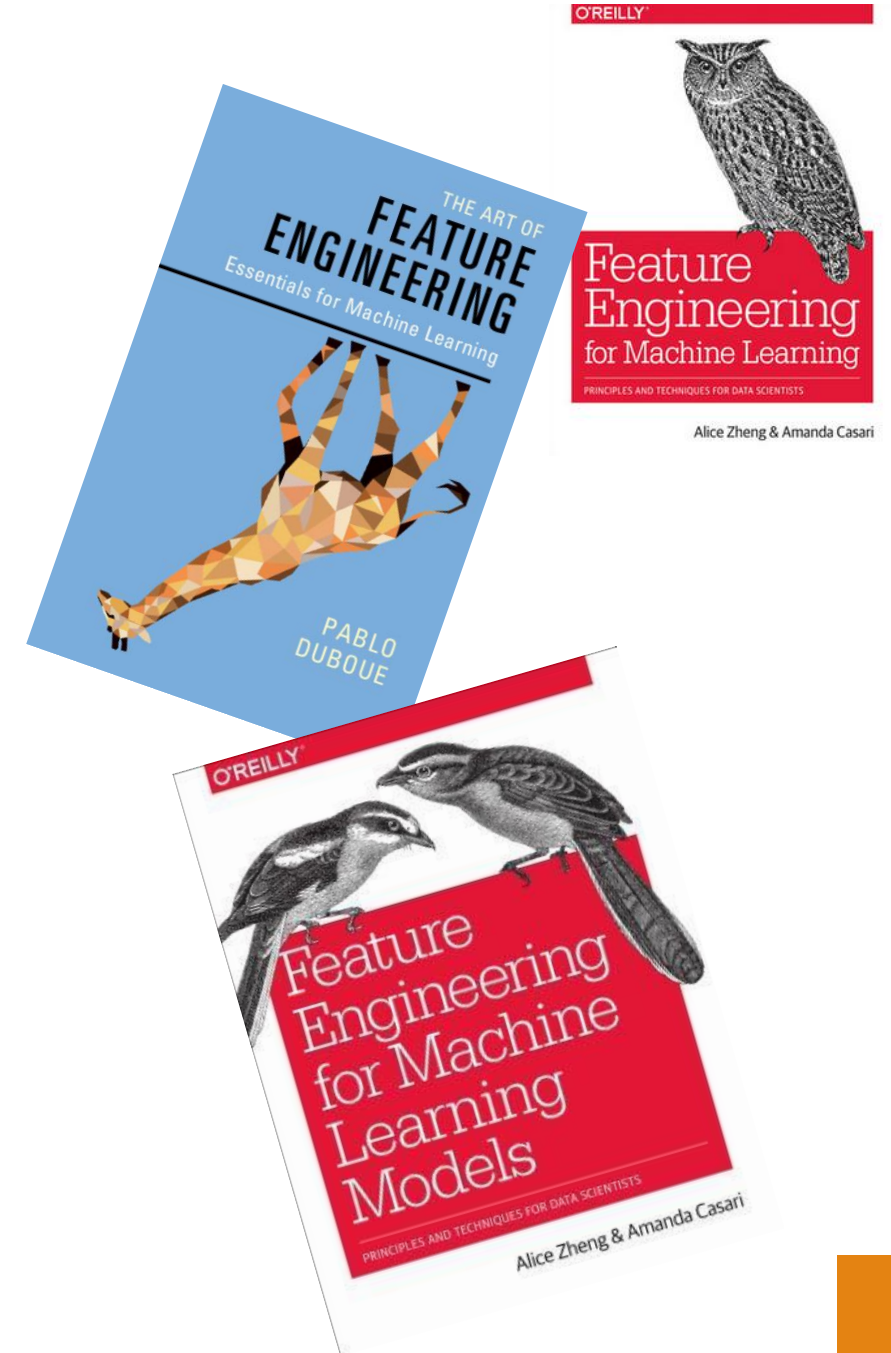
Introduction

- **Why the term “feature engineering”?**
 - A **variable** is a general statistical concept referring to any measurable attribute that can take different values across observations
 - In datasets, variables correspond to the columns of the data table and may include input variables, output variables, identifiers or metadata
 - A **feature** is a variable used as input to a machine learning model
 - Features are the variables that describe the instances and help the model make predictions
 - All features are variables; however, not all variables are features

Feature engineering

Introduction

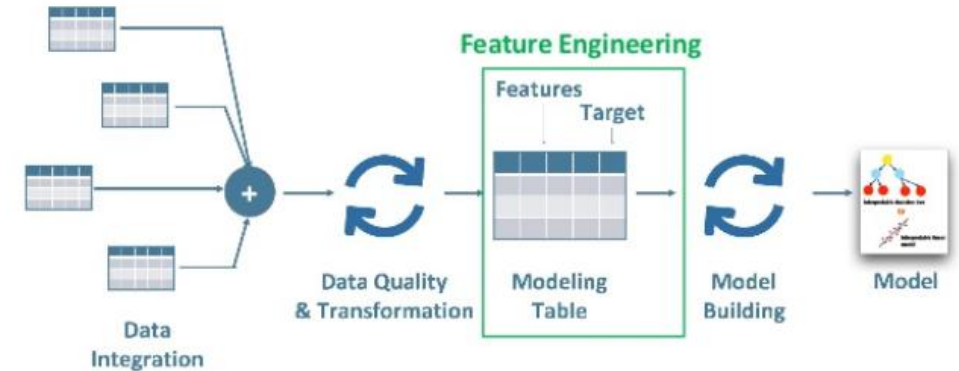
- Why the term “feature engineering”?
 - Another reason for the change in terminology is because, in ML, the term **feature** is also used because variables may be transformed or derived, or more, generally engineered
 - These derived attributes are **features**, even though they were **not original variables in the dataset**
 - **Feature engineering** is still an active research topic in Machine Learning
 - Although the nature of the research has changed compared with the early days of ML
 - The field has shifted from manual feature crafting toward automation, representation learning, and feature discovery methods



Feature engineering

Introduction

- **Feature Engineering** is the critical process of creating, transforming, and selecting variables to improve model performance
 - Includes numerous tasks that can be grouped as:
 - Feature creation
 - Feature transformation
 - Feature selection
 - Feature extraction
 - Handling class imbalance
- Dimensionality reduction

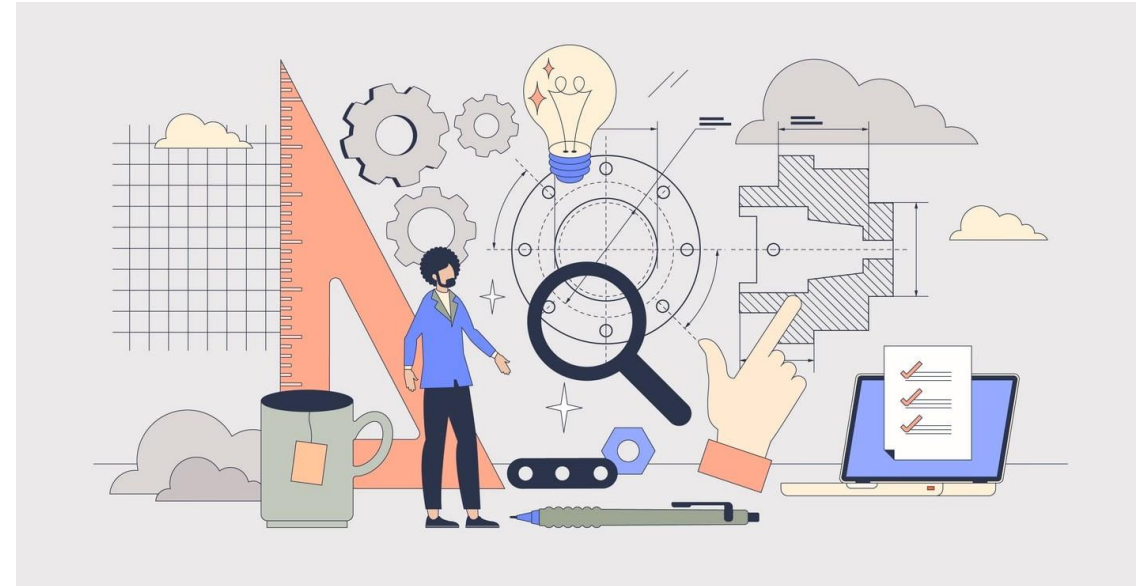


Getting Started with Feature Engineering:
<https://www.analyticsvidhya.com/blog/2020/10/getting-started-with-feature-engineering/>

Feature engineering

Feature creation

- **Feature creation** is the task of creating new features from existing data
 - Requires human intervention and creativity, often domain expertise
 - Combines existing features using operations like addition, subtraction, multiplication, and ratios
 - Creates derived features that improve model performance



What Is Feature Engineering? Techniques, Examples, and ML Use Cases:
<https://fonzi.ai/blog/feature-engineering>

Feature engineering

Feature creation

- **Typical tasks of feature creation:**
 - Simple transformations
 - Ratios and normalizations
 - Interaction terms
 - Aggregations (Group-based features)
 - Time-based / Lag features
 - Decomposition / Component extraction

Feature engineering

Feature creation – Typical tasks

- **Simple transformations** – Common in churn prediction, customer analytics, HR models
 - $\text{Age} = \text{Current Date} - \text{Birth Date}$
 - $\text{Account tenure} = \text{Current Date} - \text{Account Creation Date}$
 - $\text{Session duration} = \text{Logout Time} - \text{Login Time}$
- **Ratios and normalizations** – Useful when scale differences distort raw values
 - $\text{BMI} = \text{Weight} / \text{Height}^2$
 - $\text{Debt-to-income ratio} = \text{Total Debt} / \text{Annual Income}$
 - $\text{Conversion rate} = \text{Purchases} / \text{Website Visits}$
 - $\text{Spend per visit} = \text{Total Spend} / \text{Number of Visits}$

Feature engineering

Feature creation – Typical tasks

- **Interaction terms** – Captures effects that only appear when variables act together
 - Socioeconomic Interaction = $\text{Income} \times \text{Education Level}$
 - Life Stage Interaction = $\text{Age} \times \text{Number of Children}$
 - Channel Elasticity = $\text{Discount} \times \text{Marketing Channel}$
 - Thermal Comfort Index = $\text{Temperature} \times \text{Humidity}$ (energy demand forecasting)
- **Aggregations (Group-based features)** – Very common in transactional and behavioral data
 - Total purchases per customer
 - Average order value per user
 - Number of transactions in last 30 days
 - Mean grade per student

Feature engineering

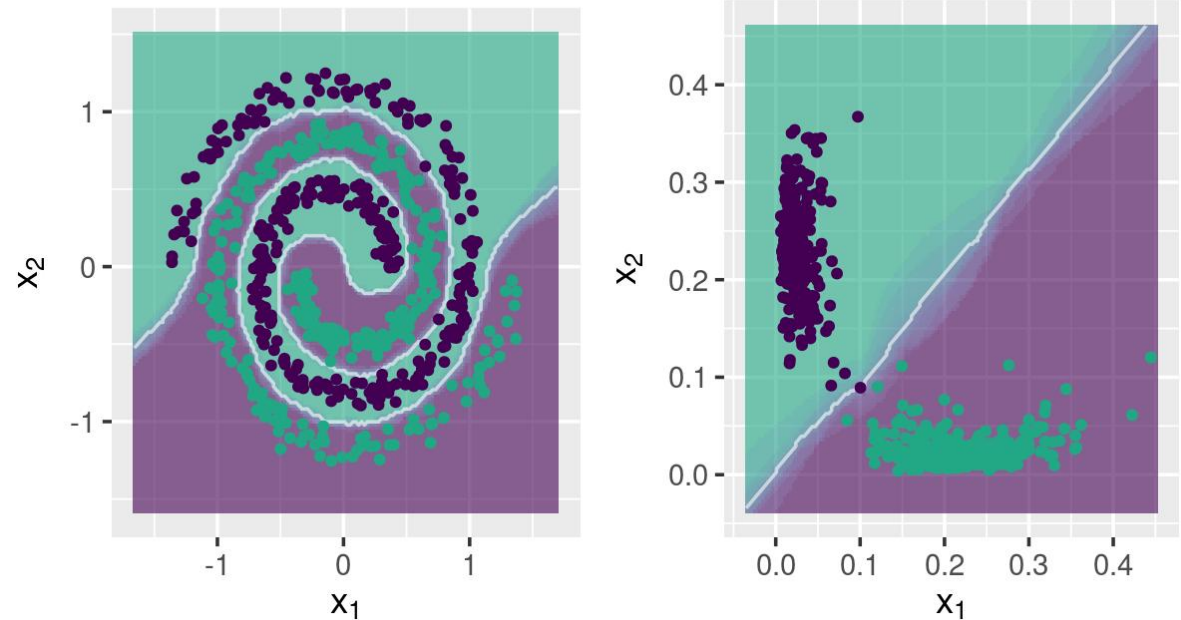
Feature creation – Typical tasks

- **Time-based / Lag features** – Essential in forecasting and sequential modelling
 - Sales yesterday
 - Sales in previous week
 - Difference from last month
 - Rolling 7-day average
 - Time since last purchase
- **Decomposition / Component Extraction** – Extracting meaningful components from existing variables
 - Date decomposition: Date $\rightarrow$ {Year, Month, Day, Weekday, IsWeekend}
 - Time decomposition: Timestamp $\rightarrow$ {Hour, Part of Day}
 - Text length features: Review $\rightarrow$ {Number of Words, Average Word Length}
 - Address parsing: Address $\rightarrow$ {City, Postal Code, Region}
 - Signal decomposition: Sensor signal $\rightarrow$ {Trend, Seasonal Component, Residual}

Feature engineering

Feature transformation

- **Feature transformation** are tasks that modify variables to make them more suitable for learning
 - Ensure model flexibility with diverse data inputs
 - Standardize feature for better interpretability
 - Improve accuracy and prevent computational errors
 - Keep features within an acceptable range for the model



Feature Extraction with KNN: <https://davpinto.github.io/fastknn/articles/knn-extraction.html>

Feature engineering

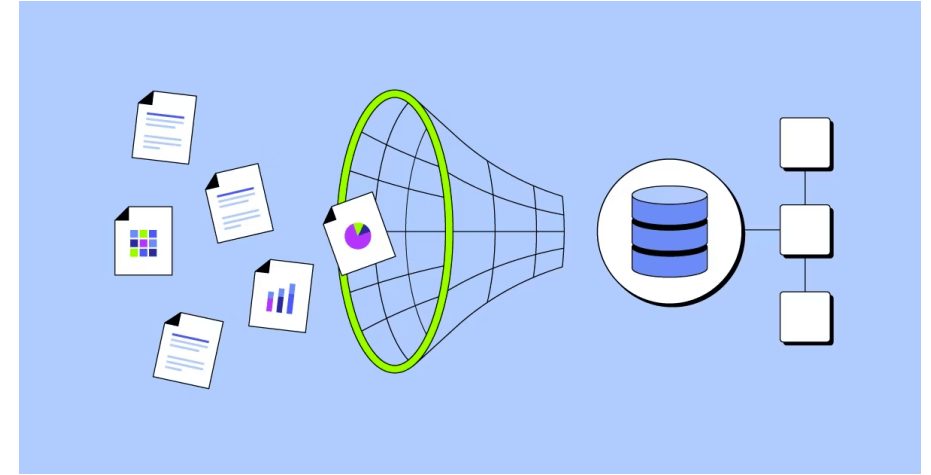
Feature transformation

- **Typical tasks of feature transformation:**
 - **Data cleaning** – Detect, correct, or remove errors in data
 - **Noise reduction** – Reduce random variations or fluctuations in the data
 - **Feature encoding** – Convert categorical/text features into numeric representations
 - **Cardinality reduction** – Reduce the number of unique categories in high-cardinality features
 - **Feature scaling** – Normalize or standardize numerical features for algorithm compatibility
 - **Distribution shaping** – Apply transformations to make distributions closer to normal
 - **Discretization (binning)** – Convert numeric variables into discrete intervals if needed

Feature engineering

Feature transformation – Data cleaning

- **Data cleaning** involves detecting, correcting, or removing errors in a dataset
- The objective is to ensure that the data is accurate, consistent, and suitable for analysis or modelling
- In the literature:
 - Some authors refer to this process also as **data wrangling**
 - Others consider data wrangling a broader process, which includes data cleaning together with data transformation and restructuring



What is Data Wrangling? Importance, Steps, and Examples:
<https://www.almabetter.com/bytes/articles/what-is-data-wrangling>

Feature engineering

Feature transformation – Data cleaning

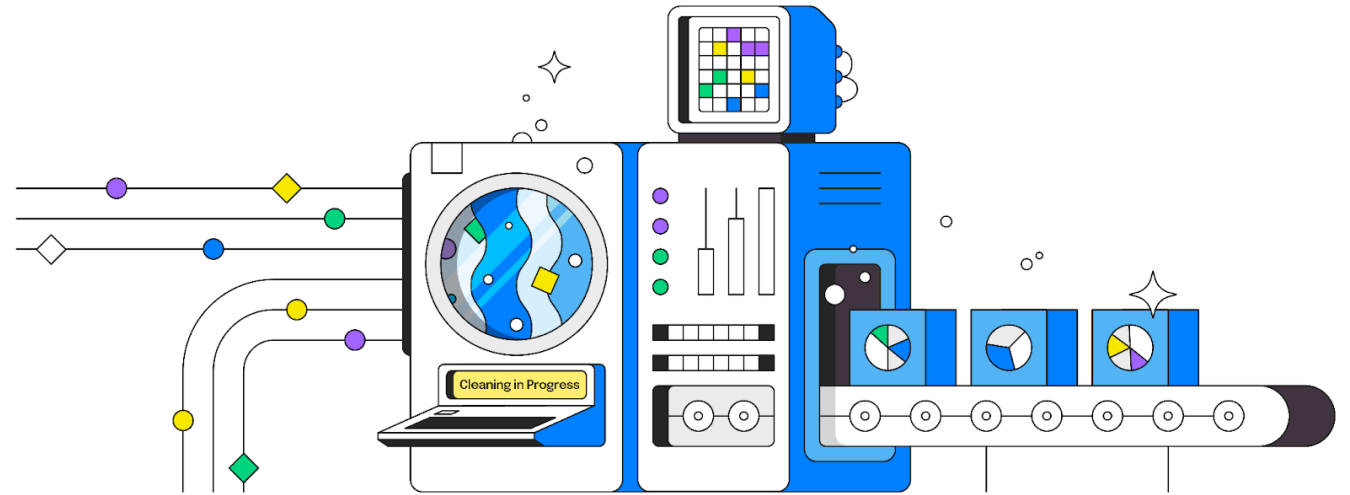
- Data cleaning is usually **not** considered feature engineering
 - Focuses on **data quality**, not creating new predictive information
 - **Operational** rather than creative as it fixes and standardizes data but does not generate new features
 - **Supports** but does not enhance model performance directly
 - Clean data **prevents errors**, but feature engineering actively improves predictions
- However, clean data is essential for effective feature engineering, without it, engineered features may be misleading or ineffective
 - For this reason, it is typically addressed in the context of feature engineering as a **preliminary task**

Feature engineering

Feature transformation – Data cleaning

■ Typical tasks of data cleaning:

- Handling data errors
- Handling duplicate data
- Handling missing data
- Handling outliers
- Handling inconsistent formats
- Handling inconsistent data types



What is Data Wrangling? Importance, Steps, and Examples:
<https://www.almabetter.com/bytes/articles/what-is-data-wrangling>

Feature engineering

Feature transformation – Data cleaning: handling data errors

- **Handling data errors** involves identifying and correcting inaccurate, inconsistent, or invalid values in a dataset introduced during data entry, data collection, transmission, or processing
- Common causes:
 - Manual input mistakes
 - Sensor malfunction
 - Encoding problems
 - Data integration from multiple systems
- Types of errors:
 - Typographical errors
 - Logical inconsistencies
 - Impossible values and domain violations



Transforming Data Entry: How to Eliminate Errors and Maximize Efficiency: <https://www.baytechconsulting.com/blog/transforming-data-entry-how-to-eliminate-errors-and-maximize-efficiency-2025>

Feature engineering

Feature transformation – Data cleaning: handling data errors

- Examples of errors in a dataset:

Attribute	Value	Problem
Age	250	Impossible value
Email	john@@mail.com	Invalid format
Salary	-5000	Logical inconsistency
Grade in course	0	Logical contradiction
Passed course	yes	
Country	Porrtugal	Typo

- Typical strategies to detect data errors:

- Validation rules
- Range checks
- Regular expressions
- Cross-field consistency check



Transforming Data Entry: How to Eliminate Errors and Maximize Efficiency: <https://www.baytechconsulting.com/blog/transforming-data-entry-how-to-eliminate-errors-and-maximize-efficiency-2025>

Feature engineering

Feature transformation – Data cleaning: handling data errors

■ Handling data errors

Error type	Description	Recommended actions	Guidelines
Impossible values	Values that violate physical, temporal, or domain constraints (e.g., negative age, future birthdate)	Correct if obvious, Recollect, Delete as last resort	Prefer correction or recollection over deletion; only delete if rare and unfixable
Invalid format	Data not matching expected format (e.g., phone, email, ID)	Correct automatically (regex, formatting), Manual review, Recollect/Delete if unfixable	Automated correction works well for predictable patterns
Logical inconsistencies	Cross-field contradictions (e.g., end date < start date)	Correct using domain rules, Recollect, Delete if unresolvable	Apply domain knowledge to resolve; deletion only if no reliable fix
Logical contradictions / constraints	Violations of domain rules (e.g., married = yes but age < 12)	Verify and correct, Recollect, Delete if uncertain	Investigate before deletion; some cases may reveal deeper issues
Typos	Minor spelling or categorical inconsistencies	Automatic correction (fuzzy matching), Manual review, Delete if unrecognizable	Often easy to fix automatically, keep logs of corrections

Feature engineering

Feature transformation – Data cleaning: handling duplicate data

- **Handling duplicate data** involves identifying and resolving repeated records within a dataset
- Types:
 - Exact duplicates
 - Near duplicates (slight variations)
- From the algorithm's perspective, duplicates are **additional evidence**, not redundancy:
 - Reweight the training distribution
 - Increase the influence of specific patterns
 - Potentially bias the induced model
- Duplicate handling is important in data cleaning, particularly when duplicates **do not** represent meaningful repeated measurements but instead arise from data integration or entry errors

Feature engineering

Feature transformation – Data cleaning: handling duplicate data

- Examples of duplicates in a dataset:

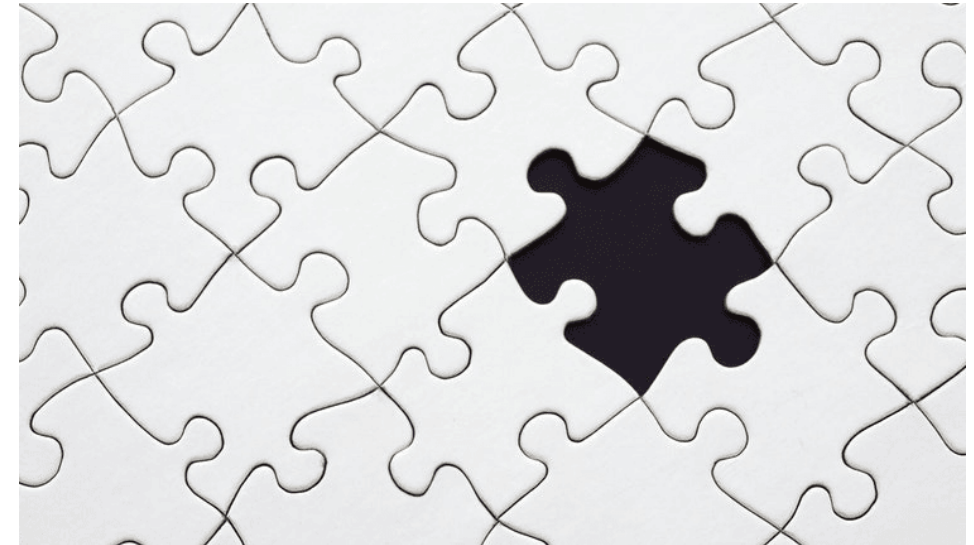
ID	Name	Email	
101	Ana Silva	ana@gmail.com	} exact duplicates
101	Ana Silva	ana@gmail.com	
102	João Pereira	joao@gmail.com	} near duplicates
102	Joao Pereira	joao@gmail.com	

- Typical strategies to handle duplicate data:
 - **Transform near duplicates into exact duplicates**
 - There are many entity resolution techniques
 - Fuzzy matching algorithms compute a similarity score between strings
 - If the score exceeds a defined threshold, the entities are considered a match
 - **Remove exact duplicates and coalesce into one**

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Handling missing data** involves identifying and appropriately treating absent values within a dataset, where one or more attribute values are not recorded for a given observation
- Typical representations:
 - Empty string
 - ?
 - NULL
 - NaN
- Practical implications:
 - Reduces dataset size
 - Introduces bias
 - Some algorithms cannot handle missing values



What is Data Wrangling? Importance, Steps, and Examples:
<https://www.almabetter.com/bytes/articles/what-is-data-wrangling>

Feature engineering

Feature transformation – Data cleaning: handling missing data

- Examples of a dataset with missing data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0.0	A/5 21171	7.2500		S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1			71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female		0	0.0	STON/O2. 3101282	7.9250		S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0.0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0.0	373450	8.0500		S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0.0	211536	13.0000		S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0.0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female		1	2.0	W./C. 6607	23.4500		S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0.0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0.0	370376	7.7500		Q

891 rows × 12 columns

- Typical strategies to handle missing data:

- Deletion methods
- Substitution methods
- Missing-value-tolerant algorithm
- Missingness indicator

	Survived	Age	Fare
0	1.0	22.0	7.250
1	NaN	NaN	NaN
2	NaN	26.0	7.925
3	NaN	NaN	53.100
4	0.0	35.0	8.050
...	...	...	...
886	0.0	27.0	13.000
887	1.0	19.0	30.000
888	0.0	NaN	23.450
889	1.0	26.0	30.000
890	0.0	32.0	7.750

891 rows × 3 columns

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Deletion methods** – remove missing data
 - **Listwise deletion** – Remove rows with missing values
 - Suitable when missing values are few
 - **Pairwise deletion** – removes selected rows with missing values
 - Suitable when it does not make sense to have two missing values
 - **Column deletion** – Remove features with excessive missing values
 - Used when a column has too many missing entries to be reliable

	Survived	Age	Fare
0	1.0	22.0	7.250
1	NaN	NaN	NaN
2	NaN	26.0	7.925
3	NaN	NaN	53.100
4	0.0	35.0	8.050
...	...	...	...
886	0.0	27.0	13.000
887	1.0	19.0	30.000
888	0.0	NaN	23.450
889	1.0	26.0	30.000
890	0.0	32.0	7.750

891 rows × 3 columns

All methods imply potential loss of useful data!

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Substitution methods** – Replace missing values with estimated values
 - **Domain-specific substitution** – Replace missing values based on **structured knowledge** about the domain rather than purely statistical rules, examples:
 - *Body temperature* – Fill missing values with the patient's **baseline temperature** recorded at admission rather than the global mean
 - *Blood pressure* – If a reading is missing during surgery, fill it with the **most recent stable intraoperative measurement**
 - *Laboratory tests* – If a test was not ordered because the physician judged it unnecessary, fill it with a **clinically normal reference value** rather than treating it as random missingness

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Substitution methods** – Replace missing values with estimated values
 - **Simple imputation** – Replace missing values by estimating/assigning values from available data
 - **Constant value** – Fills missing values with a fixed value (Example: 0, "Unknown", or "N/A")
 - Particularly useful when missing values represent an actual category ("No Response" in survey data)
 - **Central tendency value:**
 - **Mean** – Fills missing values with the mean (suitable for normally distributed numerical data)
 - **Median** – Fills missing values with the middle-point examples (preferred for skewed numerical data)
 - **Mode** – Fills missing values with the most frequent category (for categorical data)

Feature engineering

Feature transformation – Data cleaning: handling missing data

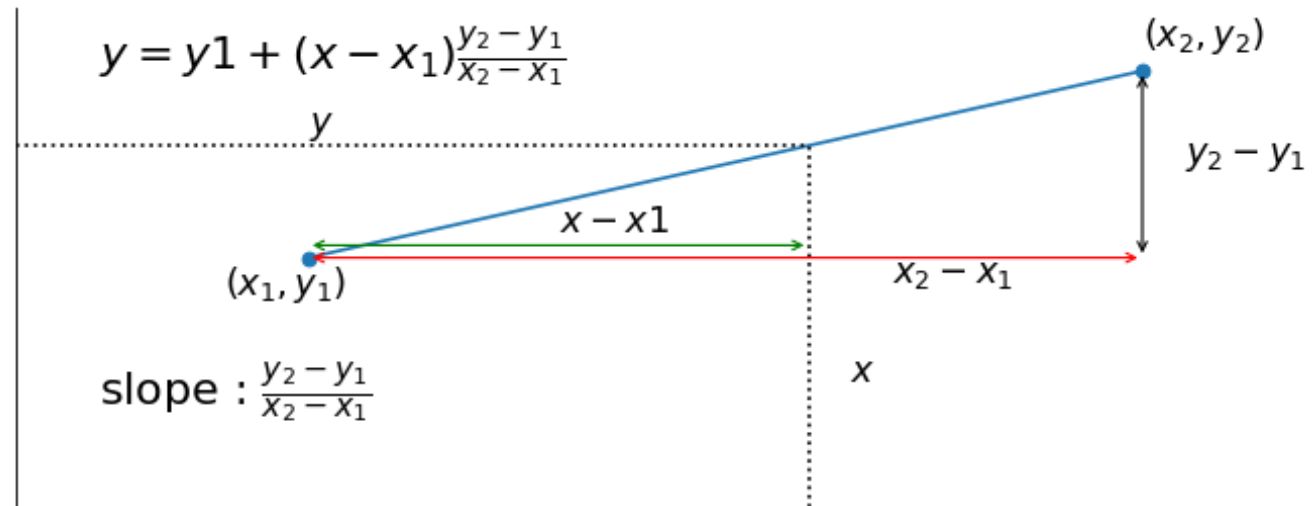
- **Substitution methods** – Replace missing values with estimated values
 - **Interpolation** – Replace missing values by estimating them from **surrounding known values**, typically in ordered or continuous data such as time series or spatial measurements
 - Relies on the assumption that data points are related in **sequence** or **space**
 - Univariate by nature
 - Common methods:
 - **Linear interpolation** – Fits a straight-line estimation between known points
 - **Spline interpolation** – Fits smooth curves passing through known points
 - **Polynomial interpolation** – Fits a polynomial function through the data

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Linear interpolation** – Fits a straight-line estimation between known points

$$y = y_1 + (x - x_1) \frac{y_2 - y_1}{x_2 - x_1}$$



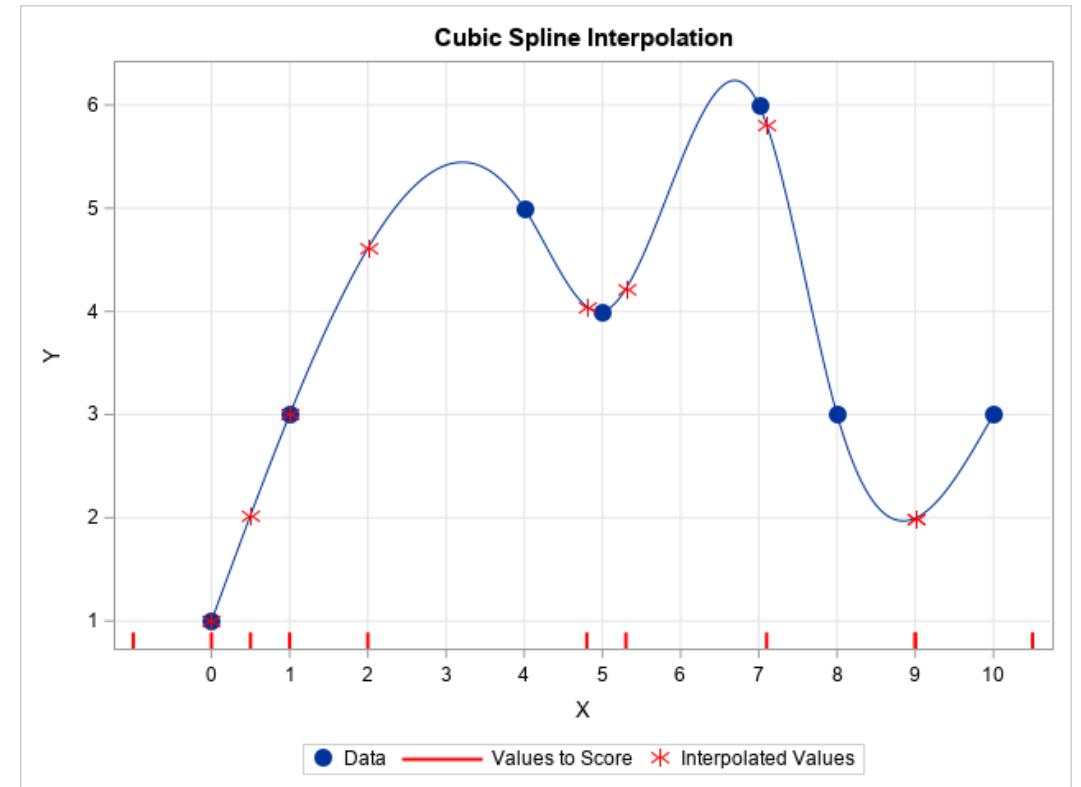
Linear interpolation: https://matthew-brett.github.io/teaching/linear_interpolation.html

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Spline interpolation** – Fits smooth curves passing through known points

$$y = a_1 + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$



Cubic spline interpolation in SAS: <https://blogs.sas.com/content/iml/2020/05/11/cubic-interpolation-sas.html>

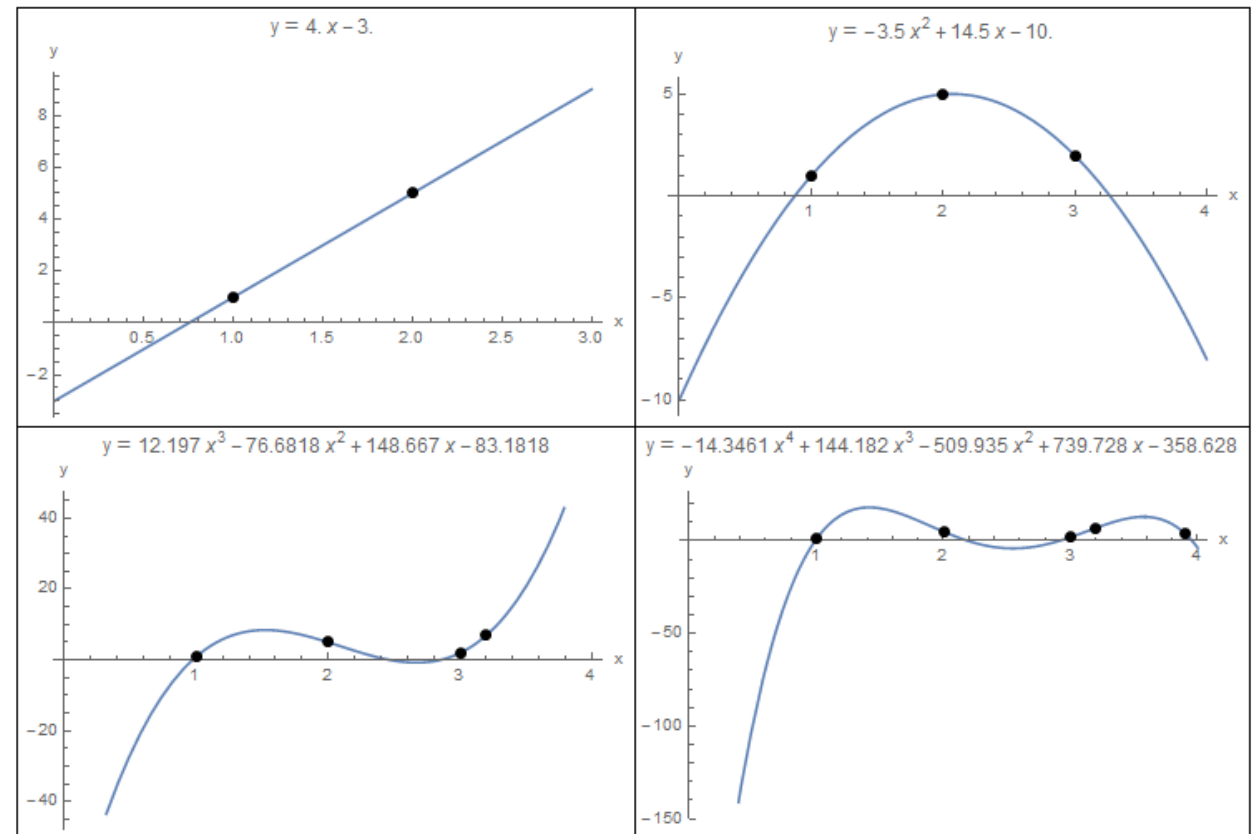
Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Polynomial interpolation** – Fits a polynomial function through the data

$$y = \sum_{i=0}^n y_i L_i(x)$$

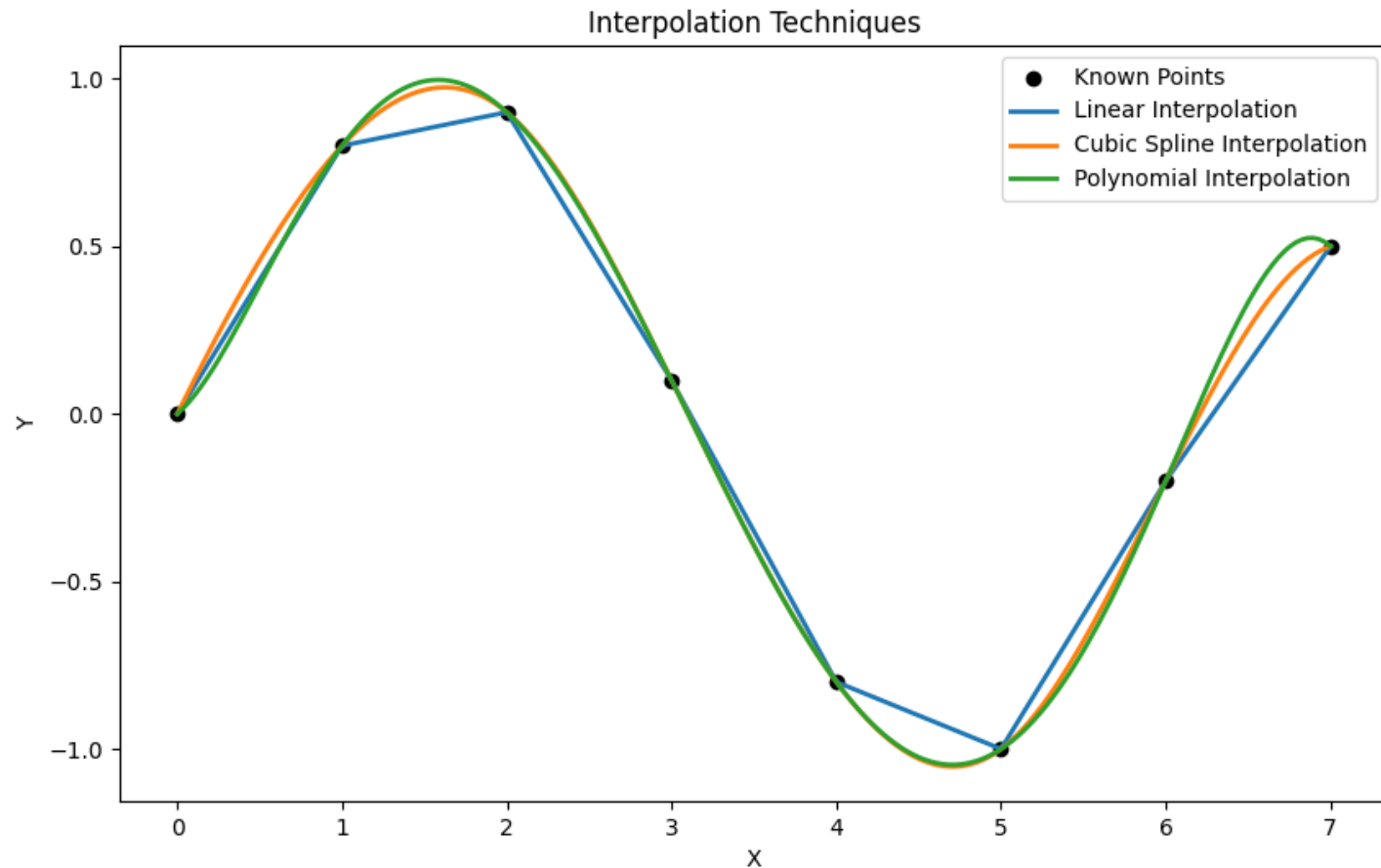
$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$



Feature engineering

Feature transformation – Data cleaning: handling missing data

■ Example of interpolation methods



Feature engineering

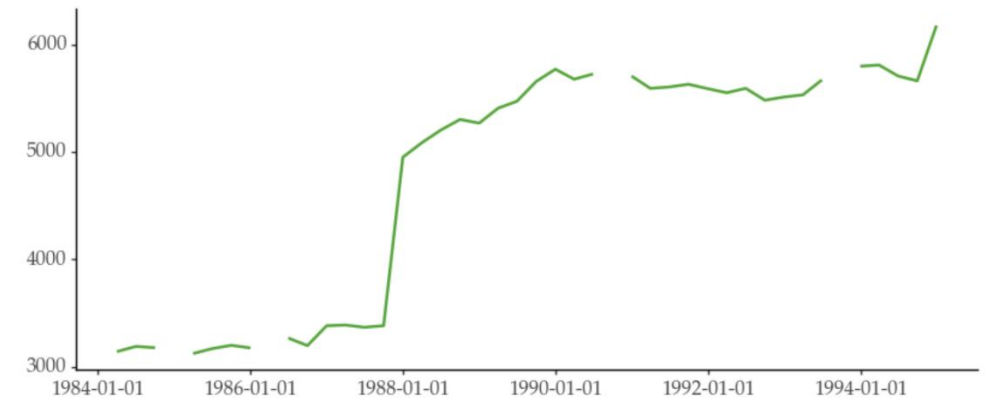
Feature transformation – Data cleaning: handling missing data

- **Substitution methods** – Replace missing values with estimated values
 - **Advanced imputation** – Replace missing values based on a prediction using a ML algorithm from all available data (using relationships between variables)
 - Multivariate by nature
 - Common methods:
 - **Regression-based imputation** – Fills missing values using a regression model trained on available data
 - Example: Using multiple linear regression to estimate a missing numerical value
 - **k-Nearest Neighbours (kNN) imputation** – Fills missing values based on the closest k observations in the dataset
 - More effective when there are similar examples

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Substitution methods** – Replace missing values with estimated values
 - **Forward/backward filling** – Techniques used to propagate nearby observed values to replace missing entries in ordered data
 - Often used for time-series data
 - Common methods:
 - **Forward Fill**
 - Replaces a missing value with the previous valid observation: $x_t = x_{t-1}$
 - Assumes short-term stability or persistence
 - **Backward Fill**
 - Replaces a missing value with the next valid observation : $x_t = x_{t+1}$
 - Assumes future observation reflects the missing period



How to Handle Time Series Missing Data: <https://towardsdatascience.com/how-to-handle-time-series-missing-data-d45e9aaae72c/>

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Forward/backward filling**
- **Advantages:**
 - Simple and computationally efficient
 - Preserves temporal structure
 - No need for model-based estimation
- **Potential disadvantages:**
 - Can propagate outdated information
 - Not suitable for long missing intervals
 - May distort trends if overused

Date	Sales
Day 1	100
Day 2	NA
Day 3	NA
Day 4	120
Day 5	115

Date	Sales
Day 1	100
Day 2	100
Day 3	100
Day 4	120
Day 5	115

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Missing-value-tolerant algorithm** – Using a machine learning models that can naturally process missing values without requiring proper handling
 - Examples:
 - **Decision Trees**
 - Can split on available features while ignoring missing values
 - Some implementations assign missing values to the most common branch
 - **k-Nearest Neighbors (kNN)**
 - Can compute distances using only available features

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Missingness indicator** – Binary variable added to the dataset to explicitly encode whether a value was originally missing

$$z_i = I[x_i \text{ is missing}] = \begin{cases} 1 & \text{if } x_i \text{ is missing} \\ 0 & \text{otherwise} \end{cases}$$

- Rationale:
 - Retains information contained in the missingness pattern
 - Enables the model to capture whether the absence of a value is itself informative
 - Reduces the risk of masking structural missingness through imputation alone
- Implementation procedure
 1. Create binary indicator for each variable with missing values
 2. Apply an imputation strategy (mean, median, kNN, regression-based, etc.)
 3. Train the model including both (imputed variable, corresponding missingness indicator)

Feature engineering

Feature transformation – Data cleaning: handling missing data

- **Missingness indicator**

Income	Income_missing
50000	0
NA	1
42000	0

If missing income is correlated with unemployment, the indicator may improve predictions

- **Advantages:**

- Missingness is informative
- Absence reflects a systematic process (e.g., not applicable, not reported)

- **Potential disadvantages:**

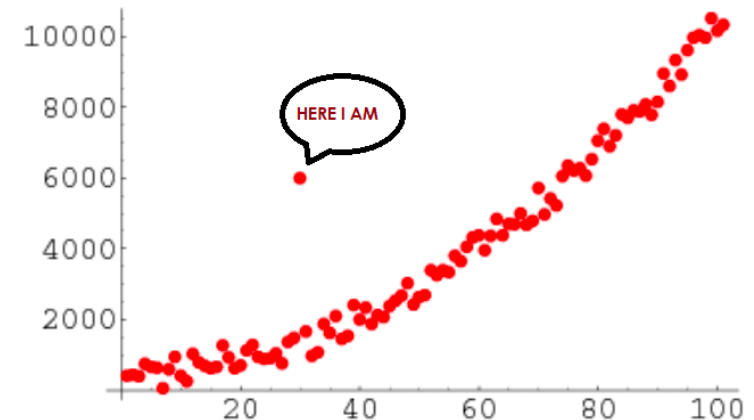
- Increases dimensionality
- May introduce noise if missingness is purely random
- Requires careful interpretation

Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Handling outliers** involves first identifying them and then applying appropriate treatment
- An **outlier** is an example that deviates significantly from most of the data
- Causes:
 - Measurement errors
 - Data entry mistakes
 - Genuine rare events – Valid but rare observations in the dataset
- Practical implications:
 - Distorts mean and variance
 - Impacts regression models
 - Influences distance-based algorithms

} Mistakes in data collection or processing



What is Data Wrangling? Importance, Steps, and Examples:
<https://www.almabetter.com/bytes/articles/what-is-data-wrangling>

Feature engineering

Feature transformation – Data cleaning: handling outliers

■ Typical strategies to detect outliers:

□ Visualization

- Scatterplots
- Boxplots

□ Statistical methods

- Z-Score

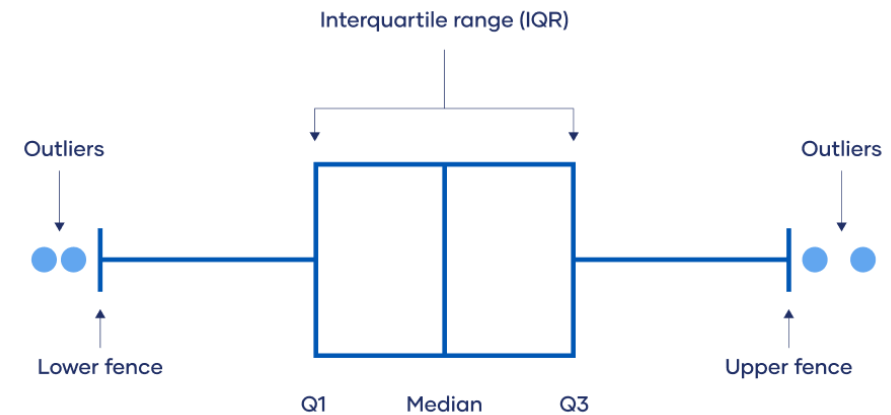
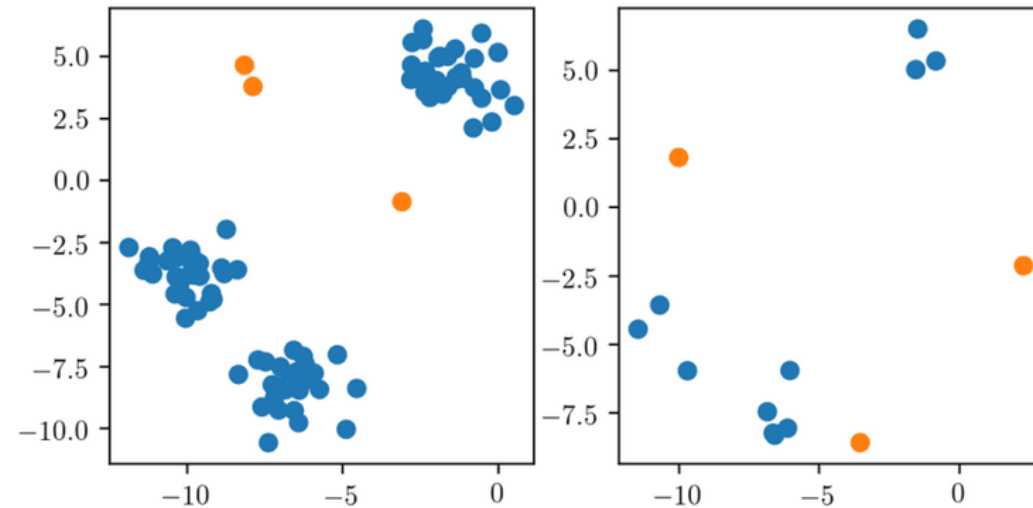
$$Z = \frac{X - \mu}{\sigma}$$

$$Z > 3 \quad \vee \quad Z < -3$$

- Interquartile range

$$IQR = Q_3 - Q_1$$

$$X < (Q_1 - 1.5 IQR) \quad \vee \quad X > (Q_3 + 1.5 IQR)$$



Feature engineering

Feature transformation – Data cleaning: handling outliers

- Examples with outliers in a dataset:

Univariate outliers

Person	Income (€)
A	1800
B	2050
C	1950
D	50000

Multivariate outliers

Height (cm)	Weight (kg)
170	65
175	70
180	75
210	140

- Typical strategies to handle outliers:
 - **Deletion**
 - **Transformation**
 - **Capping or clipping**
 - **Outlier-tolerant algorithm**

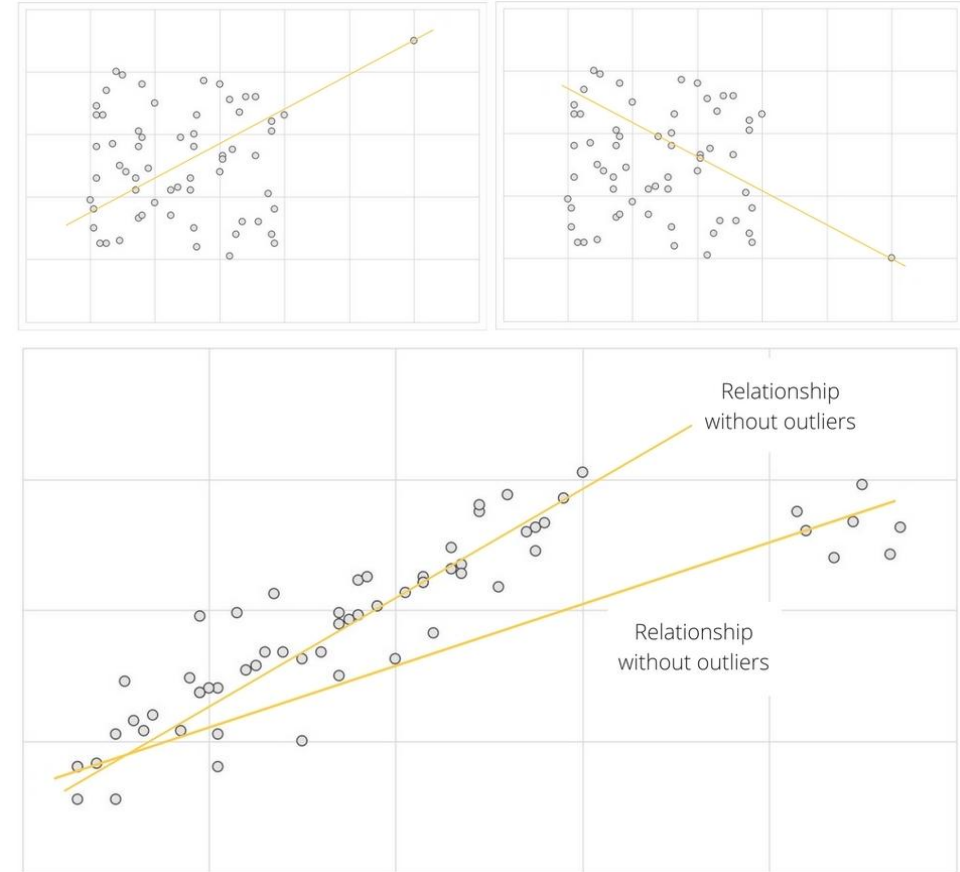
Feature engineering

Feature transformation – Data cleaning: handling outliers

■ Deletion – Remove outliers

- When the number of outliers is small
- When the outliers are likely due to errors
- Considerations:
 - Risk of losing valuable data
 - May affect the representativeness of the dataset

These outliers create a relationship where one doesn't exist otherwise. They should be deleted before the final analysis.



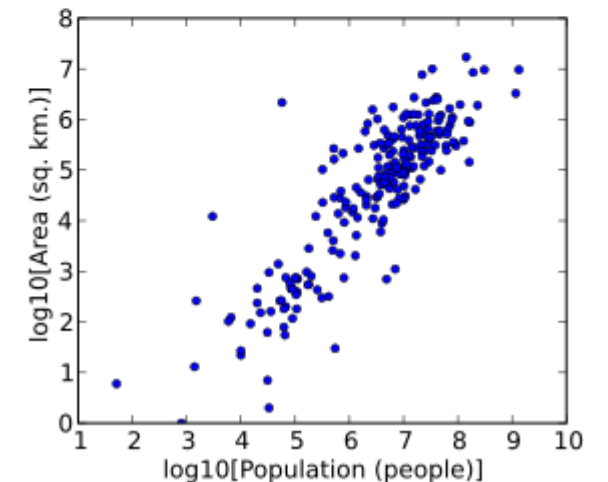
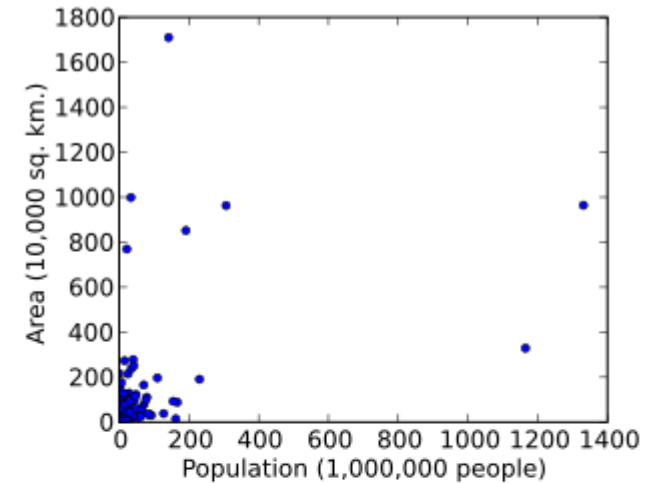
When Should You Delete Outliers from a Data Set?

<https://humansofdata.atlan.com/2018/03/when-delete-outliers-dataset/>

Feature engineering

Feature transformation – Data cleaning: handling outliers

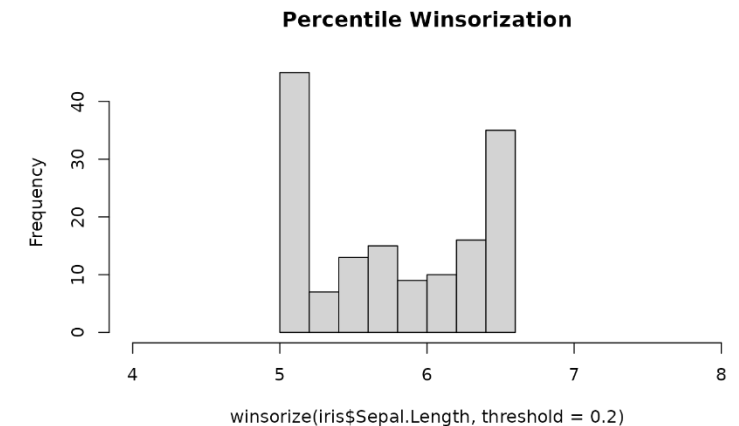
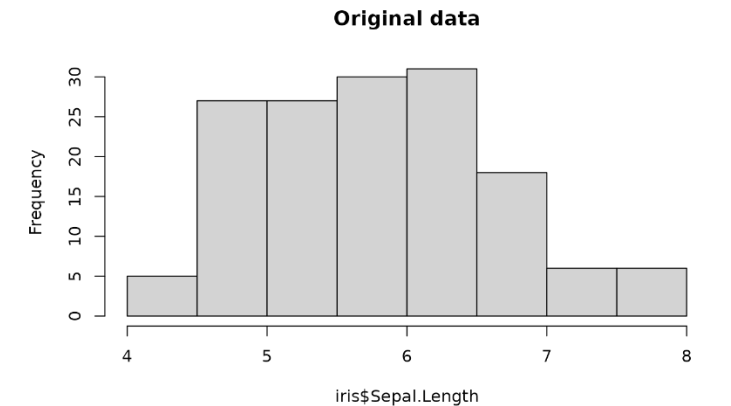
- **Transformation** – Applying transformations to reduce the impact of outliers
 - Techniques:
 - **Log transformation** – Compresses large values and reduces right skew
 - **Square root transformation** – Moderately reduces positive skew, suitable for count data
 - **Box-Cox transformation** – Parametric transformation to approximate normality for positive-valued features
 - Application of techniques of “Distribution shaping”



Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Capping or clipping** – Set thresholds to limit outlier values to a specific range
 - When it is not desirable to remove outliers but just to reduce their effect
 - Techniques:
 - **Winsorization** – Replaces outliers with a defined percentile
 - **Truncating** – Capping values beyond a threshold



Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Winsorization** is a technique that limits the influence of extreme values instead of deleting them
- Outliers are **replaced** with the nearest acceptable value within the defined range
- Typical procedure
 1. Define lower and upper percentiles (e.g., 5th and 95th percentiles)
 2. Values below the lower percentile are replaced with the lower percentile value
 3. Values above the upper percentile are replaced with the upper percentile value

Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Winsorization** example

Observation	Original	Winsorized
1	10	10
2	12	12
3	14	14
4	16	16
5	18	18
6	20	20
7	22	22
8	24	24
9	26	26
10	100	26

Winsorization using the 90th percentile

Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Truncation** limits outliers by setting a fixed threshold beyond which values are capped
- Outliers are **replaced** by a threshold
- Typical procedure
 1. Define a threshold (e.g., maximum acceptable value)
 2. Replace values above the threshold with the threshold value
 3. Optionally define a lower threshold as well

Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Truncation** example

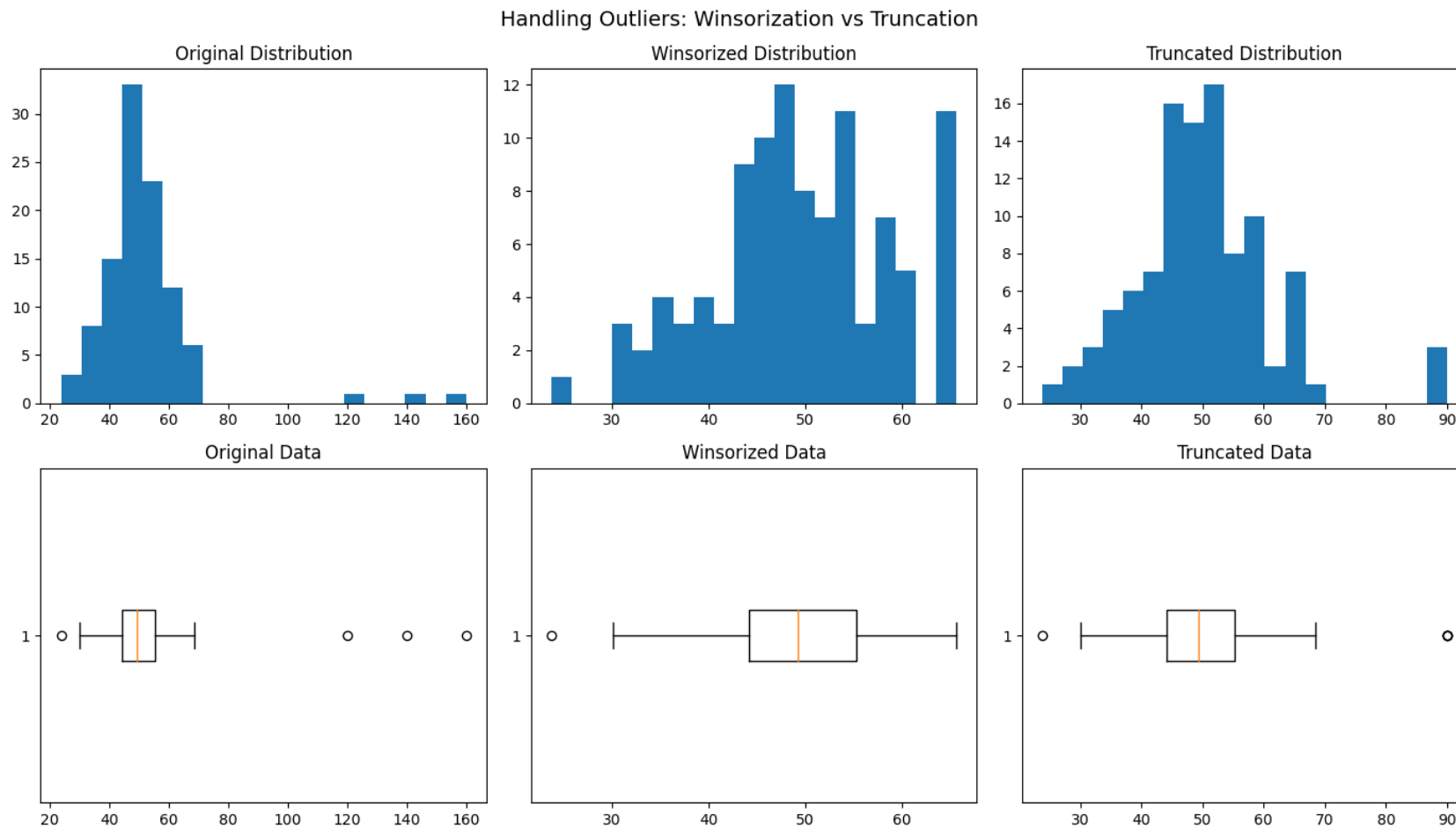
Observation	Original	Truncated
1	10	10
2	12	12
3	14	14
4	16	16
5	18	18
6	20	20
7	22	22
8	24	24
9	26	26
10	100	30

Truncation using 30 as threshold

Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Capping or clipping techniques example**



Feature engineering

Feature transformation – Data cleaning: handling outliers

- **Outlier-tolerant algorithm** – Some algorithms are more robust to outliers
 - Examples:
 - **Decision Trees and Random Forests**
 - split data based on thresholds, so outliers are covered by one branch
 - **Linear models with robust loss functions**
 - ignore examples with large errors (Huber regression, quantile regression)
 - **kNN with median aggregation**
 - use the median instead of the mean to reduce impact of extreme neighbors

Neighbors of a new house

Neighbor	Price (k€)
1	450
2	460
3	470
4	465
5 (outlier)	2000

Prediction by kNN

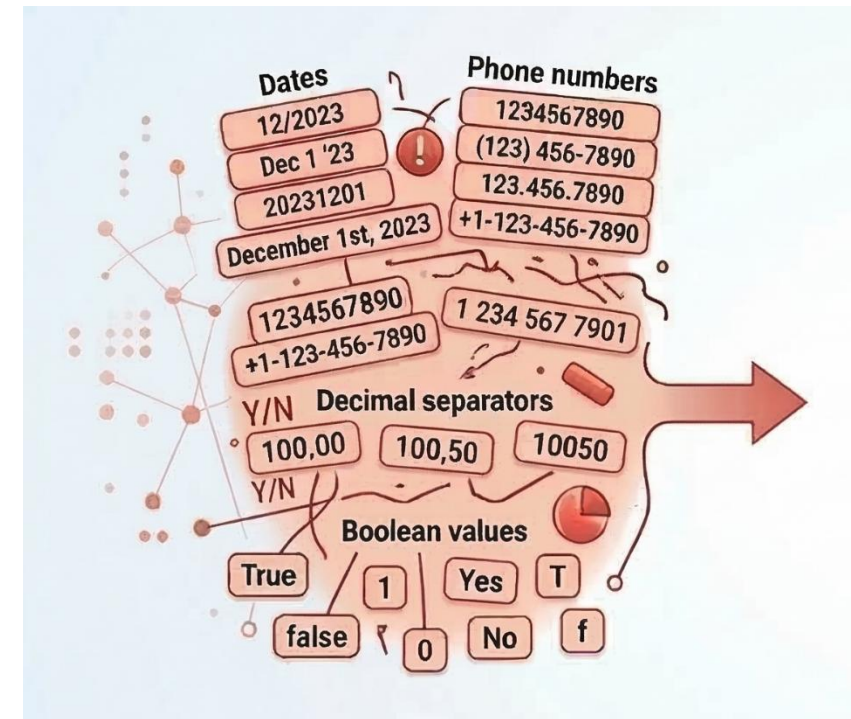
Using the mean = **769k€**

Using the median = **465k€** → more robust

Feature engineering

Feature transformation – Data cleaning: handling inconsistent formats

- **Handling inconsistent formats** involves standardizing data that is stored in different ways to ensure consistency, enabling accurate analysis and reliable model training
- Typical cases:
 - Dates
 - Phone numbers
 - Decimal separators
 - Boolean values
- Practical implications:
 - Parsing errors
 - Incorrect grouping
 - Failed joins



Feature engineering

Feature transformation – Data cleaning: handling inconsistent formats

- Examples of **format inconsistencies**:

Student ID	Data enrollment	Has scholarship	Fee
1	01/02/2024	Yes	125,00€
2	2024-02-01	0	125.00
3	Feb, 1, 2024	True	125

- Typical strategies for handling inconsistent formats:

- **Standardizing data representations** – unify formats for numbers, currencies, text, and other fields
- **Normalizing binary variables** – convert all binary values to a consistent type, preferably categorical
- **Applying international standards** – e.g., ISO 8601 for dates, ISO currency codes for financial data
- **Using pattern-based transformations** – apply regular expressions or parsing rules to enforce consistent formatting

Feature engineering

Feature transformation – Data cleaning: handling inconsistent data types

- **Handling inconsistent data types** involves ensuring that all values of a variable have the correct type, as mismatched types can lead to incorrect analysis and modeling
- Examples:
 - Numeric column stored as string
 - Mixed numeric and text values
 - Treating a categorical variable as numeric
- Practical implications:
 - Prevents numerical computation
 - Causes parsing errors
 - Affects ML pipelines

Feature engineering

Feature transformation – Data cleaning: handling inconsistent data types

- Examples of **data types inconsistencies**:

Product ID	Age	Price
1	23	19.99
2	"45"	"20"
3	"Thirty"	15€

- Typical strategies for handling inconsistent data types:

- **Type conversion** – ensuring each variable is represented in the appropriate type (numeric, categorical, datetime, etc.)
- **Conversion with specific categories** – mapping values to predefined categories to maintain consistency across the dataset
- **Data validation rules** – checking that values conform to expected types and ranges
- **Schema enforcement** – defining and enforcing a dataset schema to prevent inconsistent types during data ingestion

Feature engineering

Feature transformation – Noise reduction

- **Noise reduction** (or smoothing) is the process of reducing random variations or fluctuations in data
 - The objective is to reveal the underlying pattern or trend by reducing the influence of noise
 - It is commonly used in time series, sensor data, and experimental measurements
- **Motivation:**
 - Improve data quality before analysis or modelling
 - Make patterns and trends easier to observe
 - Reduce the impact of measurement errors or random variability
- **Typical techniques**
 - **Rolling averages** – Replace each value with the average of nearby points
 - **Signal filtering** – Remove high-frequency noise while keeping trends
 - **Exponential smoothing** – Weighted averages emphasizing recent observations

Feature engineering

Feature transformation – Noise reduction

- **Rolling averages** – Replace each value with the average of nearby points
 - Daily measurements of a variable
- Interpretation
 - The rolling average smooths short-term fluctuations
 - Instead of abrupt changes, the smoothed series changes more gradually
 - This makes the underlying trend easier to identify

Day	Value	3-Day Rolling Average
1	10	—
2	12	—
3	15	12.33
4	14	13.67
5	18	15.67
6	20	17.33
7	19	19.00
8	22	20.33
9	21	20.67

Feature engineering

Feature transformation – Noise reduction

- **Signal filtering** – Remove high-frequency noise while keeping trends
 - Daily measurements of a variable
- Interpretation
 - Signal filtering removes high-frequency fluctuations (noise) while retaining slower trends
 - Abrupt spikes caused by random variations are smoothed out, leaving the underlying trend clearer
 - Commonly used filters include low-pass filters which allow “slow” trends to pass while damping fast oscillations

Day	Value	Low-pass Filtered
1	10	10
2	12	11
3	15	12
4	14	13
5	18	14
6	20	16
7	19	17
8	22	18
9	21	20

Feature engineering

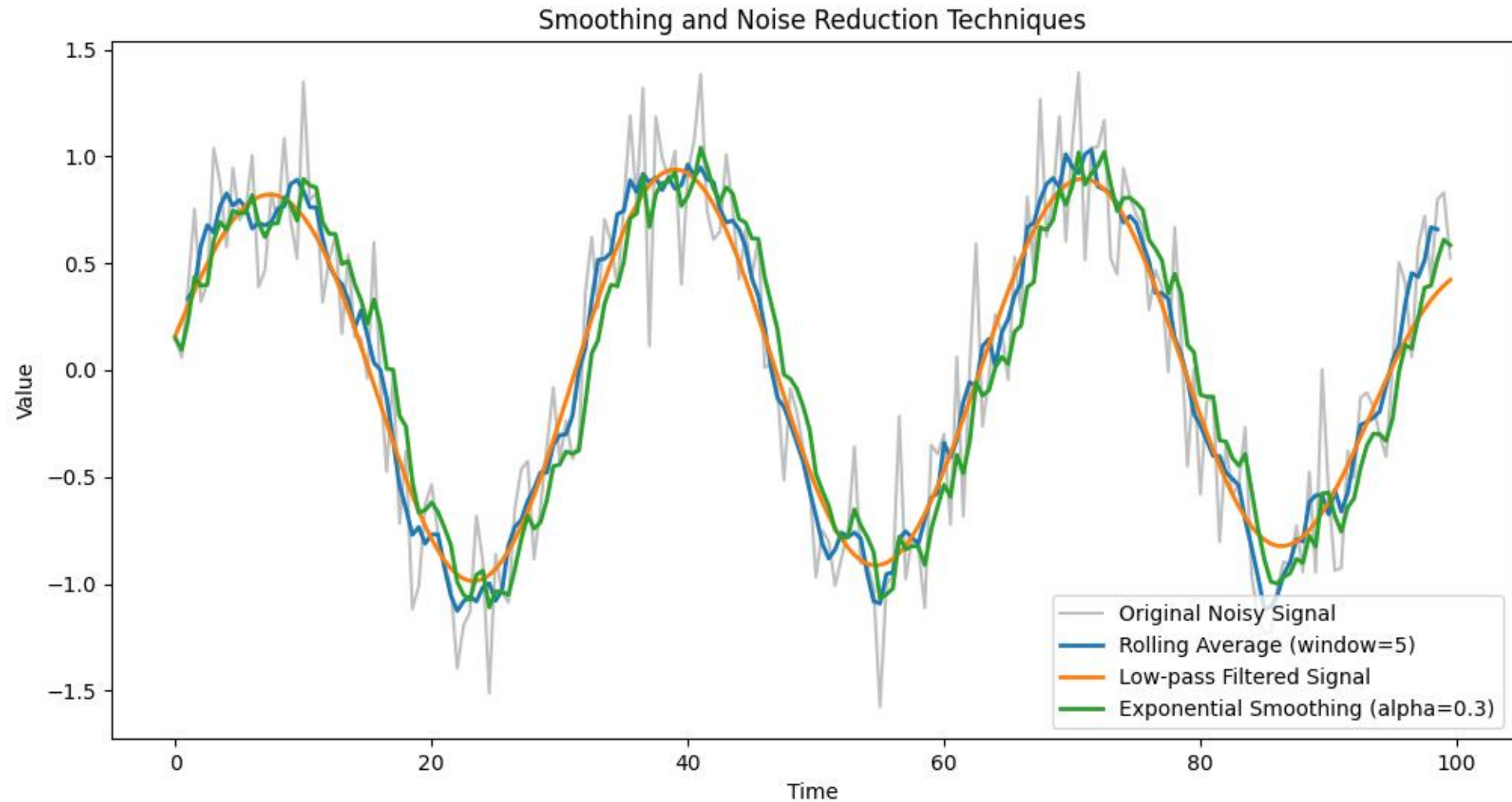
Feature transformation – Noise reduction

- **Exponential smoothing** – Weighted averages emphasizing recent observations
 - Daily measurements of a variable
- Interpretation
 - Exponential smoothing applies a weighted average where recent values have more influence
 - Smooths fluctuations while quickly adapting to changes in the trend
 - Useful for forecasting and highlighting trends in noisy data

Day	Value	Exponential Smoothed ($\alpha=0.3$)
1	10	10.00
2	12	10.60
3	15	11.92
4	14	12.74
5	18	14.42
6	20	16.49
7	19	17.49
8	22	18.85
9	21	19.59

Feature engineering

Feature transformation – Noise reduction



Feature engineering

Feature transformation – Feature encoding

- **Feature encoding** is the process of converting non-numerical data into a numerical representation that can be used by machine learning algorithms
- This transformation is necessary because most machine learning algorithms:
 - Operate on numerical vectors
 - Perform mathematical operations such as distance calculations, dot products, or gradient optimization
 - Cannot directly interpret categorical labels or text values
- The numerical representation may introduce **assumptions** about relationships between categories
 - For example, some encodings may imply an ordering or simplify category distinctions

Feature engineering

Feature transformation – Feature encoding

- Typical strategies for feature encoding:
 - **Label encoding** – Assigns a unique integer to each category
 - **Ordinal encoding** – Maps ordered categories to integers reflecting their rank
 - **One-hot encoding** – Creates a separate binary column for each category (1 if present, 0 otherwise)
 - **Target encoding** – Replaces each category with a statistic derived from the target
 - **Binary encoding** – Converts categories to integers, then to binary digits across multiple columns
 - **Hash encoding** – Uses a hash function to map categories into a fixed number of columns

Feature engineering

Feature transformation – Feature encoding

- **Label encoding** – Assigns a unique integer to each category

Color	Encoded
Red	0
Blue	1
Green	2

- **The model may assume artificial ordering!** Green > Blue > Red
- It is acceptable for tree-based models (Decision Trees, Random Forest) when order does not influence splits

Feature engineering

Feature transformation – Feature encoding

- **Ordinal encoding** – Maps ordered categories to integers reflecting their rank

Satisfaction	Encoded
Low	1
Medium	2
High	3

- **Distance has semantic meaning:** Low < Medium < High
- It is using label encoding when ordering is real and meaningful

Feature engineering

Feature transformation – Feature encoding

- **One-hot encoding** – Creates a separate binary column for each category (1 if present, 0 otherwise)

One-hot encoding

Color	Color_Red	Color_Blue	Color_Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1



Dummy encoding

Color	Color_Red	Color_Blue
Red	1	0
Blue	0	1
Green	0	0



Effect encoding

Color	Color_Red	Color_Blue
Red	1.000	0.000
Blue	0.000	1.000
Green	1.366	1.366

- Properties:

- No artificial order

- Distance is well-defined and consistent: $dist = \sqrt{2}$

- Limitations:

- Increases dimensionality

Feature engineering

Feature transformation – Feature encoding

- **Target encoding** – Replaces each category with a statistic derived from the target (e.g., mean)

School of Origin	Dropout (target)	Encoded_School_of_Origin
A	1	0.67
B	0	0.50
A	0	0.67
C	1	0.50
B	1	0.50
C	0	0.50
A	1	0.67

- The mappings used in training must be made available for testing (otherwise risks data leakage!)
- Only one numeric column is added, regardless of the number of categories

Feature engineering

Feature transformation – Feature encoding

- **Binary encoding** – Converts categories to integers, then to binary digits across multiple columns

School of Origin	Integer	Bit1	Bit2	Bit3	Bit4
A	1	0	0	0	1
B	2	0	0	1	0
C	3	0	0	1	1
D	4	0	1	0	0
E	5	0	1	0	1
F	6	0	1	1	0
G	7	0	1	1	1
H	8	1	0	0	0
I	9	1	0	0	1
J	10	1	0	1	0
K	11	1	0	1	1
L	12	1	1	0	0
M	13	1	1	0	1
N	14	1	1	1	0
O	15	1	1	1	1
P	16	0	0	0	0

- Compared to one-hot encoding, which would require 16 columns, this only uses 4 columns
- Distances are not uniform!
- Need to enforce effect encoding

Feature engineering

Feature transformation – Feature encoding

- **Hash encoding** – Uses a hash function to map categories into a fixed number of columns

School	Hash bucket (0–7)
A	3
B	5
C	1
D	7
E	0
F	3
G	2
H	6
I	4
J	1
K	0
L	5
M	2
N	6
O	7
P	3



School	H0	H1	H2	H3	H4	H5	H6	H7
A	0	0	0	1	0	0	0	0
B	0	0	0	0	0	1	0	0
C	0	1	0	0	0	0	0	0
D	0	0	0	0	0	0	0	1
E	1	0	0	0	0	0	0	0
F	0	0	0	1	0	0	0	0
G	0	0	1	0	0	0	0	0
H	0	0	0	0	0	0	1	0
I	0	0	0	0	1	0	0	0
J	0	1	0	0	0	0	0	0
K	1	0	0	0	0	0	0	0
L	0	0	0	0	0	1	0	0
M	0	0	1	0	0	0	0	0
N	0	0	0	0	0	0	1	0
O	0	0	0	0	0	0	0	1
P	0	0	0	1	0	0	0	0

Feature engineering

Feature transformation – Feature encoding

Encoding method	Advantages	Disadvantages	Best use case
Label Encoding	Simple; requires 1 column only	Imposes artificial order if categories are nominal; may mislead distance-based models	Ordinal features where order matters (e.g., Low, Medium, High)
One-Hot Encoding	No artificial order; interpretable	Number of columns grows linearly with categories; memory inefficient for high cardinality	Nominal features with few categories
Ordinal Encoding	Simple; preserves order	Assumes linear relationship between codes and outcome; not suitable for nominal features	Features with clear order (e.g., education level)
Target Encoding	Captures relationship with target; compact	Risk of target leakage; can overfit small categories; not interpretable	Features correlated with target; high cardinality
Binary Encoding	Reduces dimensionality compared to one-hot; unique encoding	May create artificial distance relationships; less interpretable	High-cardinality features; tree or linear models
Hash Encoding	Fixed vector size; memory efficient	Collisions possible; not interpretable	Very high-cardinality features; datasets too large for one-hot

Feature engineering

Feature transformation – Cardinality reduction

- **Cardinality reduction** applies to categorical variables with many unique values, which can increase model complexity, memory usage, and risk of overfitting
- Examples of high cardinality categorical variables:
 - ZIP codes / Postal codes
 - Product categories or Subcategories
 - Occupation titles / Job roles
 - Cities / Locations
 - Manufacturer / Brand names

Feature engineering

Feature transformation – Cardinality reduction

- Typical strategies for cardinality reduction:
 - **Group similar categories** – Combine similar business-criteria categories into fewer
 - **Multi-level grouping** – Combine categories into the next business hierarchical level
 - **Rare category grouping** – Combine low-frequency categories into an “Other” group
 - **Frequency encoding** – Replace each category with its frequency (or relative frequency) in the dataset
 - **Feature hashing** – Map categories into a fixed number of bins using a hash function
 - **Regularized target encoding** – Replace each category with a smoothed estimate of the target mean

Feature engineering

Feature transformation – Cardinality reduction

- **Group similar categories** – Combine similar business-criteria categories into fewer

Original categories	Reduced categories
Men's Running Shoes, Women's Running Shoes, Trail Running Shoes, Marathon Shoes, Jogging Shoes	Running Shoes
Men's Casual Shoes, Women's Casual Shoes, Slip-ons, Loafers, Moccasins	Casual Shoes
Men's Sandals, Women's Sandals, Flip-flops, Slides, Beach Sandals	Sandals
Men's Boots, Women's Boots, Hiking Boots, Work Boots, Winter Boots	Boots
Men's Formal Shoes, Women's Formal Shoes, Dress Shoes, Oxfords, Heels	Formal Shoes
...	

Feature engineering

Feature transformation – Cardinality reduction

- **Multi-level grouping** – Combine categories into the next business hierarchical level

Original categories	State (Reduced categories)	Region (Reduced categories)
New York, Buffalo, Albany	New York	East Coast
Boston, Springfield, Worcester	Massachusetts	East Coast
Los Angeles, San Francisco, San Diego	California	West Coast
Houston, Dallas, Austin	Texas	South
Miami, Orlando, Tampa	Florida	South
Denver, Boulder, Colorado Springs	Colorado	West
...		

Feature engineering

Feature transformation – Cardinality reduction

- **Rare category grouping** – Combine low-frequency categories into an “Other” group

Original categories	Reduced categories
Data Scientist, ML Engineer, AI Researcher, Software Developer, System Architect	Tech
Accountant, Financial Analyst, Auditor, Controller, Tax Specialist	Finance
Nurse, Doctor, Surgeon, Pharmacist, Therapist	Healthcare
Teacher, Professor, Lecturer, School Administrator, Tutor	Education
Artist, Musician, Designer, Historian, Writer	Other

Feature engineering

Feature transformation – Cardinality reduction

- **Frequency encoding** – Replace each category with its frequency (or relative frequency) in the dataset

Original categories	Count	Relative frequencies (Reduced categories)
New York	12	0.12
Los Angeles	9	0.09
Chicago	8	0.08
Houston	6	0.06
Miami	5	0.05
Orlando	5	
Tampa	5	
Dallas	4	0.04
Fort Worth	4	

Feature engineering

Feature transformation – Cardinality reduction

- **Feature hashing** – Map categories into a fixed number of bins using a hash function

Original categories	Hash value	Bin (hash % 5) (Reduced categories)
Nike	872391	1
Adidas	129382	2
Puma	238472	3
Reebok	918273	0
Under Armour	546372	1
New Balance	437291	4
Converse	721839	2
Vans	982374	0
Skechers	234872	3
Timberland	563728	4
Dr. Martens	817263	1
Red Wing	912837	0

Feature engineering

Feature transformation – Cardinality reduction

- **Regularized target encoding** – Replace each category with a smoothed estimate of the target mean

ZIP Code	Count	Average House Price
94103	10	1200000
94107	5	1180000
10001	50	950000
10011	30	970000
60601	3	850000
60611	4	830000
30301	2	700000
30303	1	720000
90001	6	600000
90002	5	610000
...		



ZIP Code	Smoothed target encoding	Encoded group (Reduced categories)
94103	1195000	High-end
94107	1183000	High-end
10001	948000	Mid-high
10011	964000	Mid-high
60601	846000	Mid
60611	835000	Mid
30301	710000	Mid-low
30303	725000	Mid-low
90001	610000	Low
90002	615000	Low
...		

Feature engineering

Feature transformation – Cardinality reduction

Strategy	Pros	Cons	Best use case
Group similar categories	Reduces number of categories; preserves meaningful business logic	Requires domain knowledge; subjective	When similar categories can be meaningfully merged
Multi-level grouping	Leverages existing business hierarchy; reduces categories systematically	Depends on availability of hierarchy; may lose fine-grained info	Features with a natural hierarchical structure
Rare category grouping	Handles low-frequency categories; reduces noise	May merge distinct categories into “Other”; loss of detail	High-cardinality features with many infrequent categories
Frequency encoding	Compact representation; preserves information about occurrence	May introduce spurious correlations; not interpretable	Features where frequency is predictive of target
Feature hashing	Fixed vector size; memory efficient; handles very high cardinality	Collisions possible; not interpretable	Very high-cardinality features where one-hot is infeasible
Regularized target encoding	Captures relation with target; reduces overfitting for small categories	More complex; requires smoothing; risk of leakage if not careful	Features correlated with target, especially high-cardinality categories

Feature engineering

Feature transformation – Feature scaling

- **Feature scaling** is the process of normalizing the range of numerical independent variables of data
- Example:
 - Features with different scales: e.g. "Age" (0–100) vs. "Annual Income" (20000–2000000)
 - The larger numbers will dominate the model's math, even if they are not more important
- Practical implications:
 - Critical for distance-based algorithms (e.g., kNN, k-means, SVM)
 - Essential for gradient-based methods (e.g., linear/logistic regression, neural networks)
 - Recommended after data augmentation (e.g., SMOTE) to maintain consistent scales

Feature engineering

Feature transformation – Feature scaling

- Techniques for data scaling of numerical variables:

- **Min-Max scaling** – Scales features to map the minimum to 0 and the maximum to 1

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- x_{min} – minimum observed value of the feature
 - x_{man} – maximum observed value of the feature
 - Based on a known **domain** (business-oriented domain) or the **empirical domain** (observed values in a dataset)

- **Standard scaling (Z-Score)** – Scales feature to center around 0 with $\sigma = 1$

$$x' = \frac{x - \mu}{\sigma}$$

- μ – mean of the feature
 - σ – standard deviation of the feature
 - Based on the **empirical domain** (observed values in a dataset)

Feature engineering

Feature transformation – Feature scaling

Age

$$x_{min} = 0$$

$$x_{max} = 100$$

$$\mu \approx 50$$

$$\sigma \approx 25$$

Income

$$x_{min} = 20000$$

$$x_{max} = 2000000$$

$$\mu \approx 825000$$

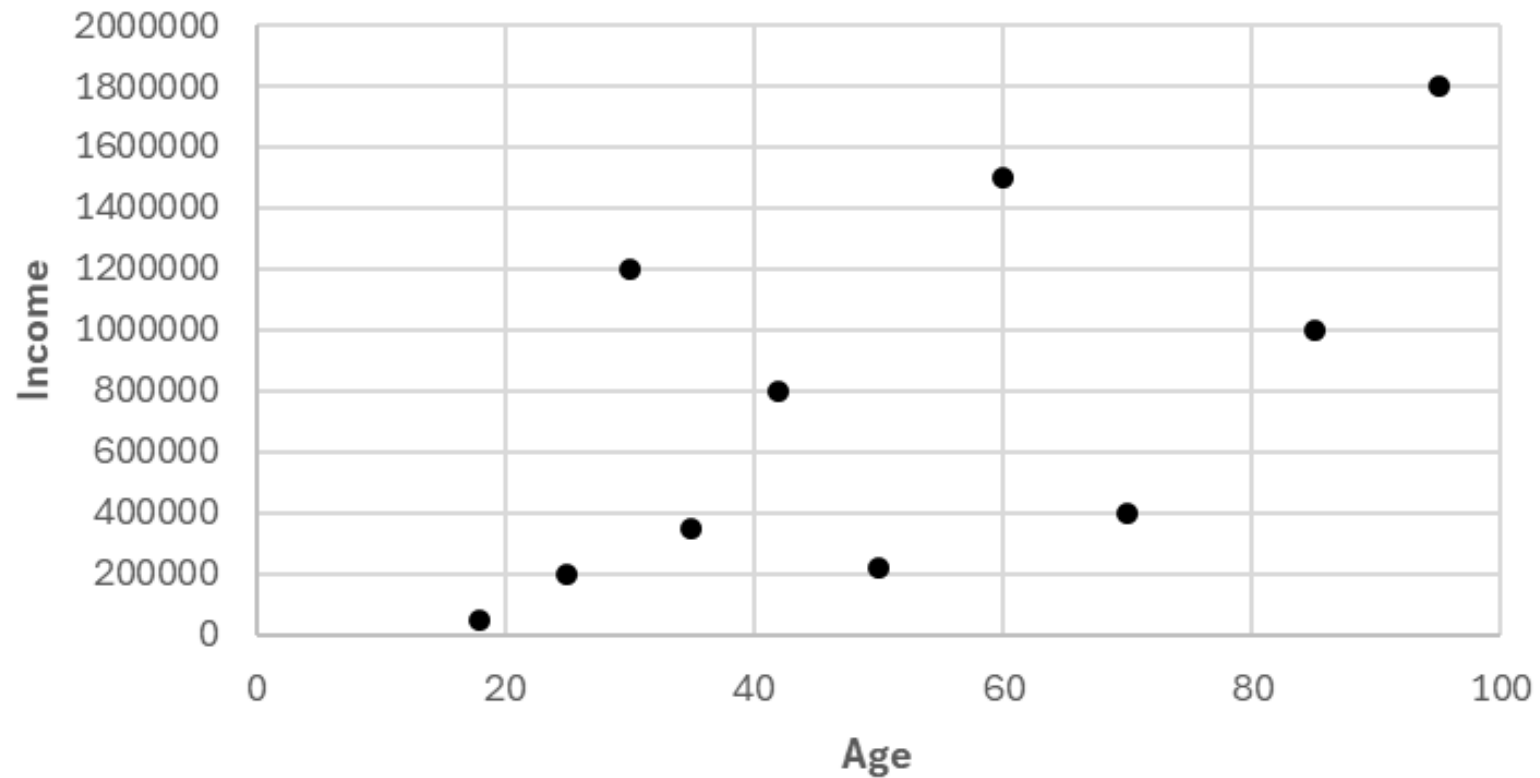
$$\sigma \approx 618000$$

Example #	Age	Income (€)	Age (Min-Max)	Income (Min-Max)	Age (Z-score)	Income (Z-score)
1	18	50000	0.18	0.015	-1.48	-1.08
2	25	200000	0.25	0.095	-1.13	-0.61
3	30	1200000	0.30	0.575	-0.88	0.71
4	35	350000	0.35	0.168	-0.63	-0.17
5	42	800000	0.42	0.394	-0.38	0.36
6	50	220000	0.50	0.100	-0.13	-0.59
7	60	1500000	0.60	0.738	0.37	0.91
8	70	400000	0.70	0.190	0.62	-0.07
9	85	1000000	0.85	0.492	1.12	0.60
10	95	1800000	0.95	0.879	1.62	1.13

Feature engineering

Feature transformation – Feature scaling

Original data

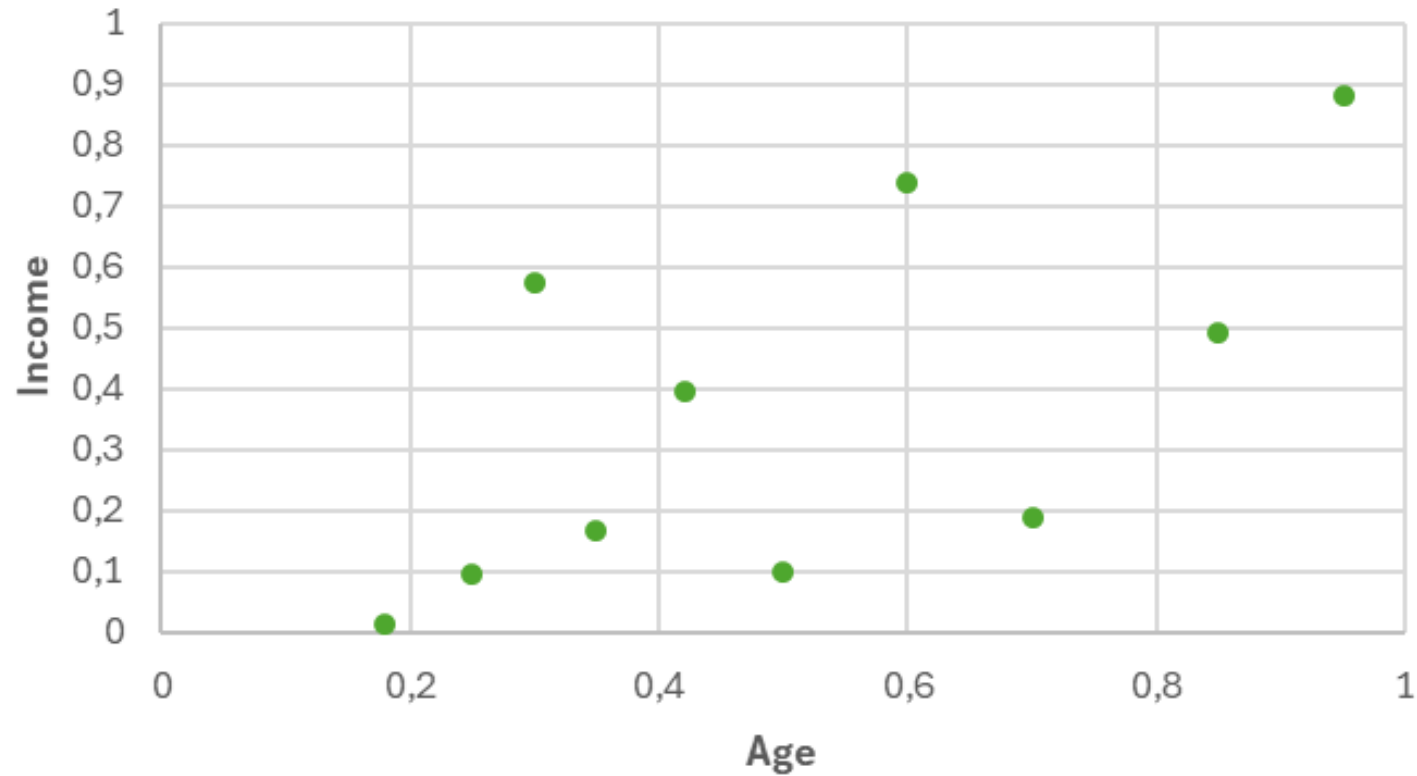


Feature engineering

Feature transformation – Feature scaling

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Min-Max scaling

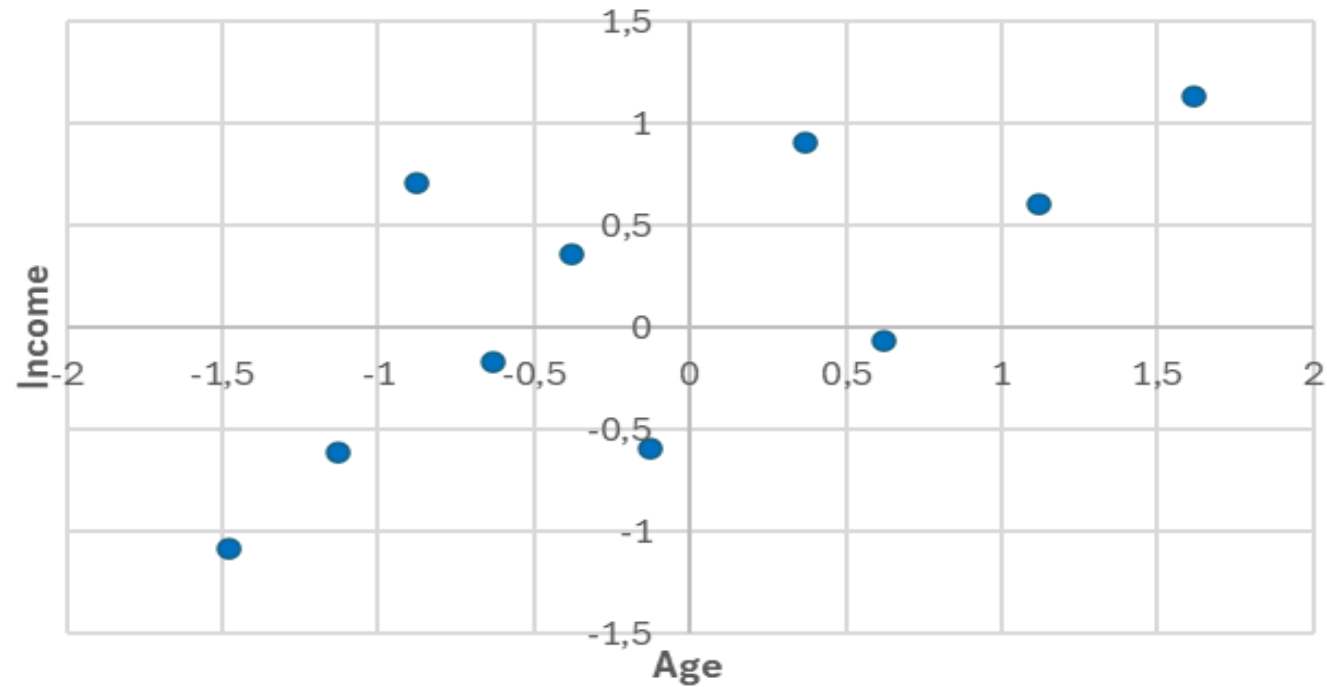


Feature engineering

Feature transformation – Feature scaling

$$x' = \frac{x - \mu}{\sigma}$$

Standard scaling



Feature engineering

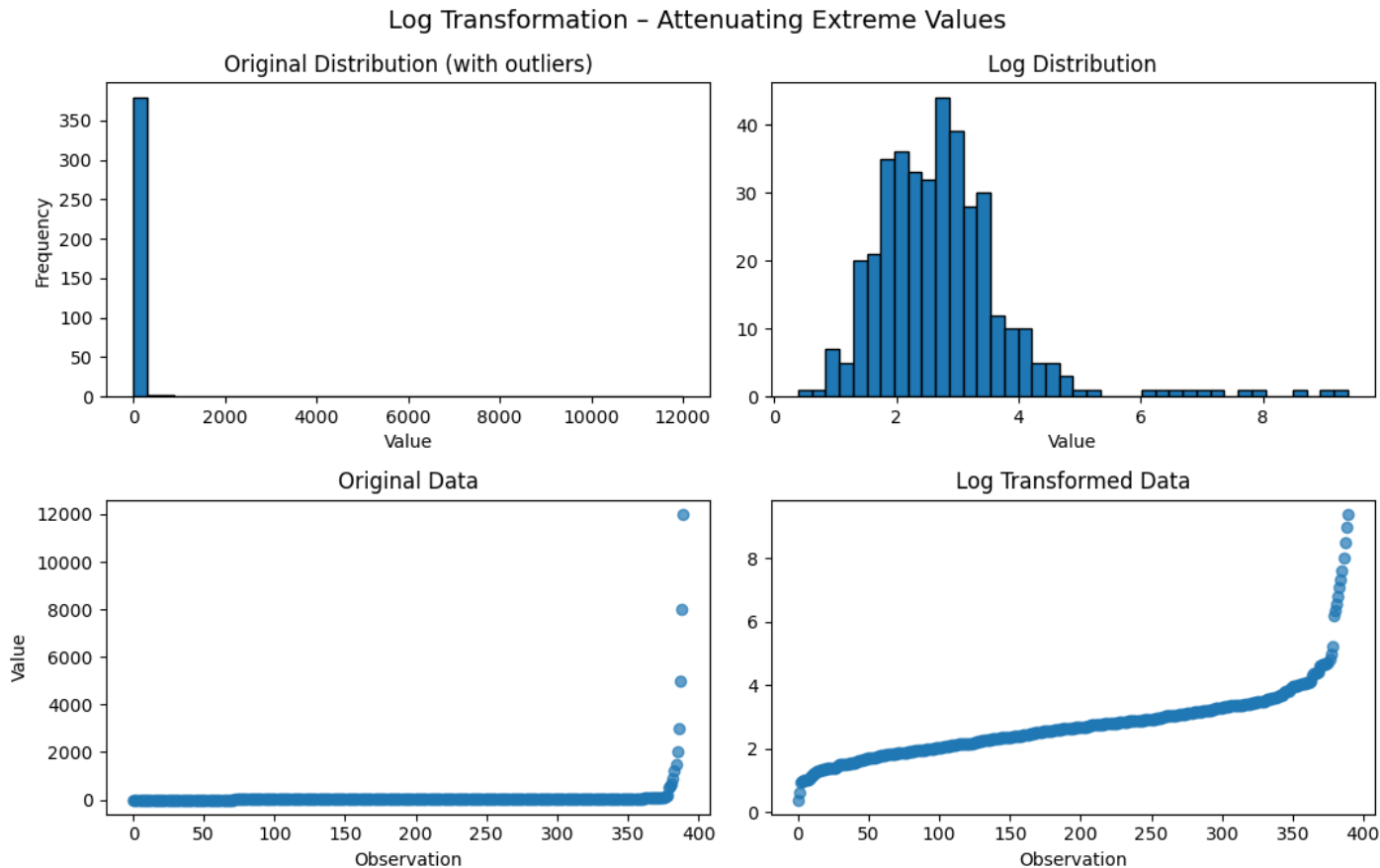
Feature transformation – Distribution shaping

- **Distribution shaping** is the process of applying mathematical transformations to numerical features to:
 - Reduce skewness
 - Mitigate the impact of extreme values (outliers)
 - Improve compatibility with algorithms that assume approximate normality
- Common transformations:
 - **Log transformation** – Compresses large values; effective for right-skewed variables
 - **Square root transformation** – Moderately reduces positive skew; suitable for count data
 - **Box-Cox / Yeo-Johnson transformations** – Parametric transforms to approximate normality (Box-Cox requires positive values; Yeo-Johnson works with zero or negative values)
 - **Power transformations** – Generalized method to adjust skewness

Feature engineering

Feature transformation – Distribution shaping

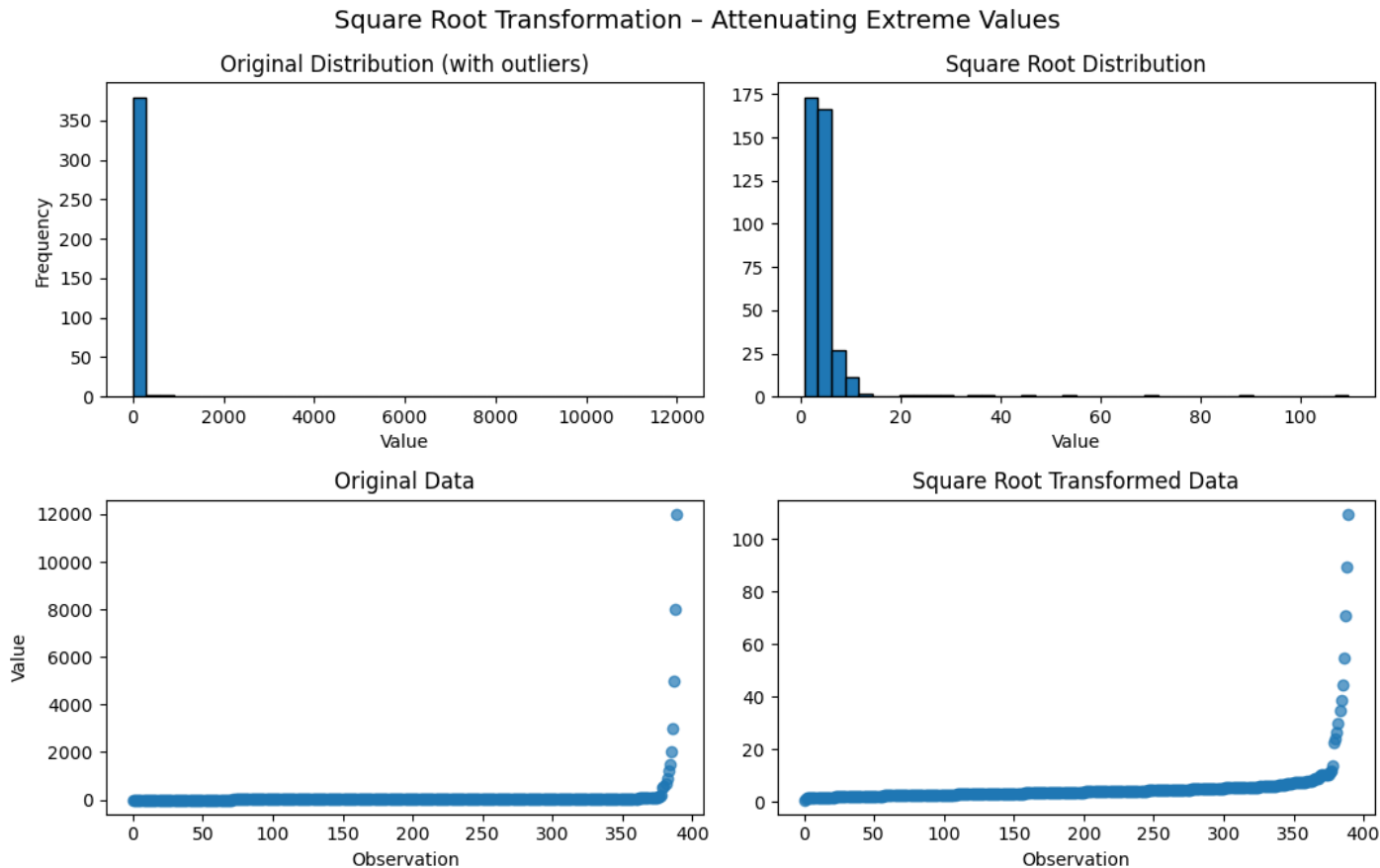
- **Log transformation** – Compresses large values; effective for positive right-skewed variables (e.g., income)



Feature engineering

Feature transformation – Distribution shaping

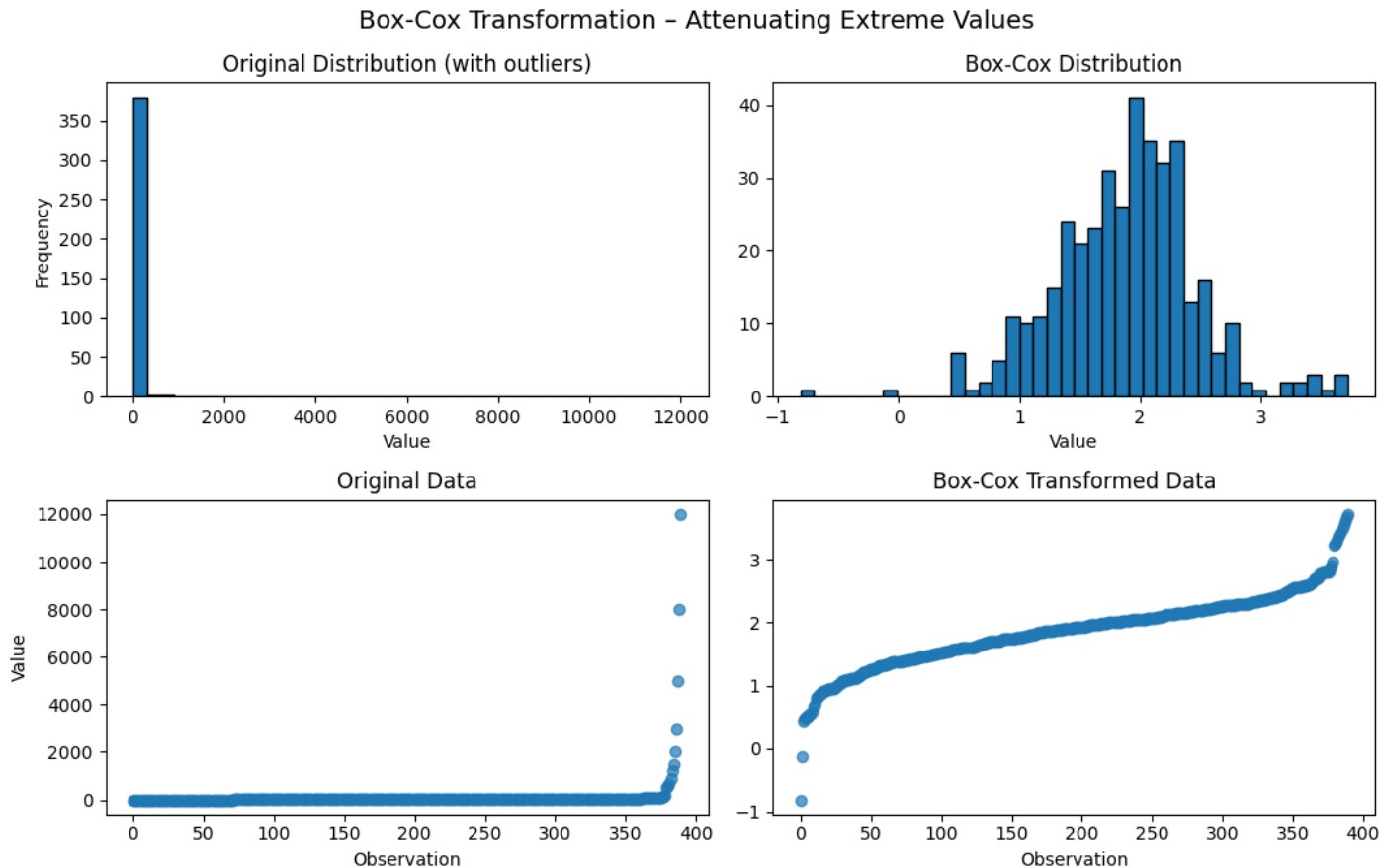
- **Square root transformation** – Moderately reduces positive skew; suitable for count data



Feature engineering

Feature transformation – Distribution shaping

- **Box-Cox transformations** – Parametric transforms to approximate normality (Box-Cox requires positive values)



Feature engineering

Feature transformation – Distribution shaping

- **Power transformations** – are a generalization of the transformations

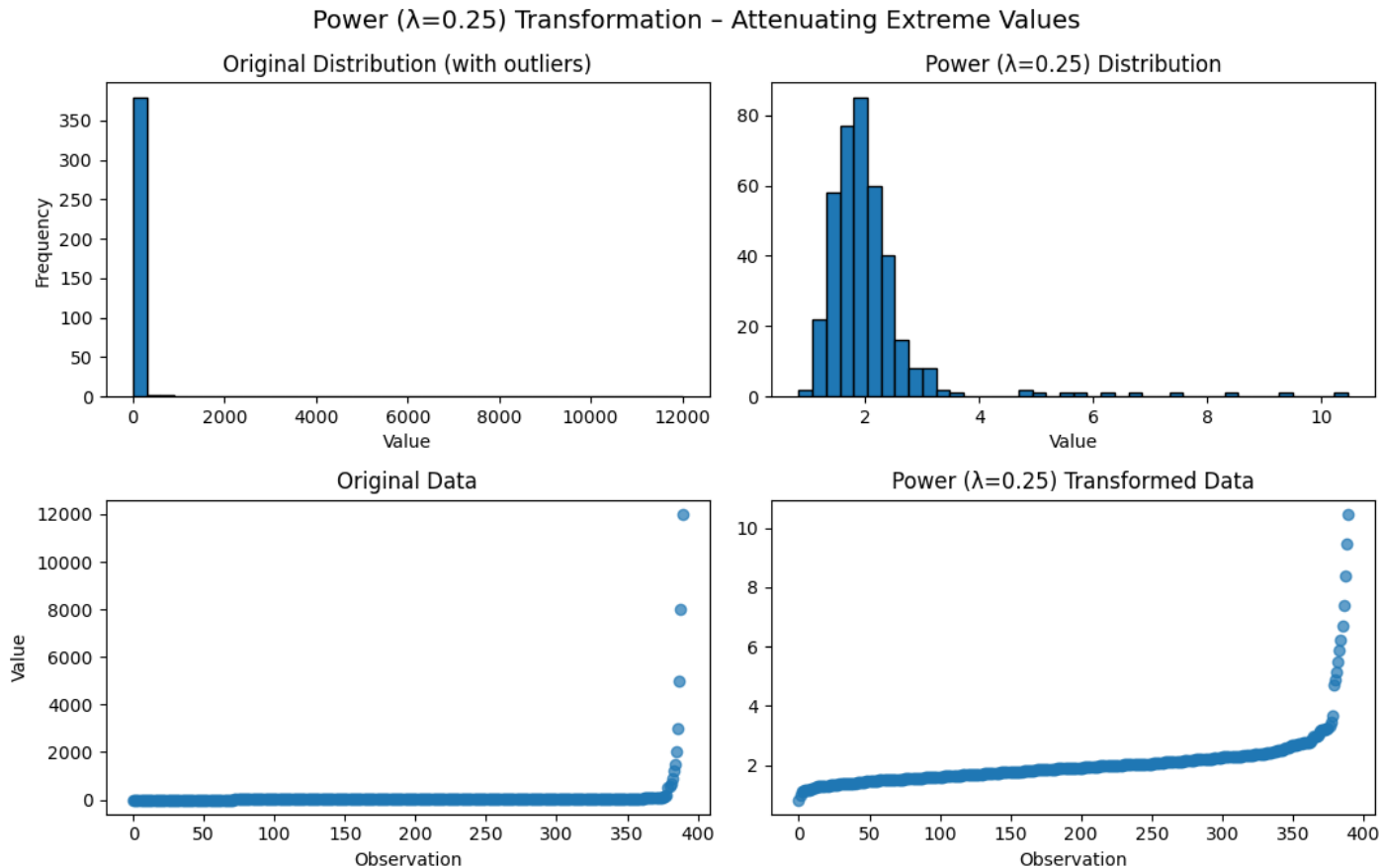
λ	Effect
$\lambda = 1$	No change
$\lambda = 0.5$	Square root
$\lambda = 0$	Log transformation ($\log(x)$)
$\lambda < 0$	Reciprocal, strong compression of large values

- Box-Cox and Yeo-Johnson are **parametric power transformations**: they automatically select the best λ to approximate normality

Feature engineering

Feature transformation – Distribution shaping

- **Power transformations** – Generalized method to adjust skewness



Feature engineering

Feature transformation – Discretization (binning)

- **Discretization** (or binning) is the process of transforming a numeric variable into a set of discrete intervals (bins)
- Instead of keeping the exact numeric value, the feature is represented by the interval to which the value belongs
- It is often applied to:
 - Simplify the representation of data
 - Improve interpretability of models
 - Reduce sensitivity to noise
 - Support algorithms that prefer categorical inputs
 - Highlight meaningful ranges of values

Feature engineering

Feature transformation – Discretization (binning)

- **Typical discretization strategies:**
 - **Equal-width binning** – The numeric range is divided into bins of identical width
 - **Quantile-binning** – Bins are created so that each bin contains approximately the same number of examples
 - **Domain-based thresholds** – Bins are defined using expert knowledge or business rules

Feature engineering

Feature transformation – Discretization (binning)

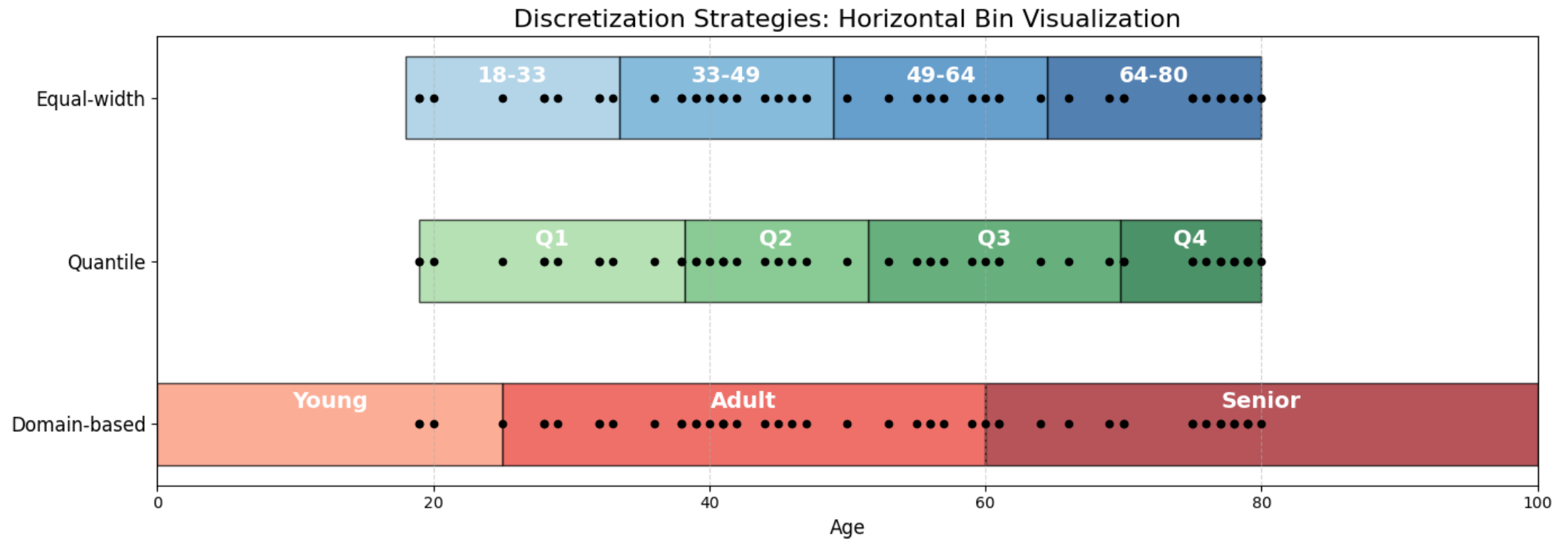
- Examples of discretization

Age	Equal-Width Bins	Quantile Bins	Domain-Based Thresholds
18	18–31	Q1 (18–25)	Young (<25)
22	18–31	Q1 (18–25)	Young (<25)
24	18–31	Q2 (25–35)	Young (<25)
30	18–31	Q2 (25–35)	Adult (25–60)
35	32–45	Q2 (25–35)	Adult (25–60)
40	32–45	Q3 (36–50)	Adult (25–60)
50	46–60	Q3 (36–50)	Adult (25–60)
60	46–60	Q4 (51–80)	Adult (25–60)
70	61–80	Q4 (51–80)	Senior (>60)
80	61–80	Q4 (51–80)	Senior (>60)

Feature engineering

Feature transformation – Discretization (binning)

- Examples of discretization



Feature engineering

Feature transformation – Discretization (binning)

- Discretization is a form of **Symbolic Data Analysis**

Single-valued data

Student ID (x_1)	Course (x_2)	Age (x_3)	Gender (x_4)	Marital status (x_5)	Distance (x_6)
1	Econometrics	19	Female	Single	2
2	Geometry	20	Male	Single	1
3	Anatomy	35	Male	Married	4



Symbolic data

Course (x_2)	Age (x'_3)	Gender (x'_4)	Marital status (x'_5)	Distance (x'_6)
Econometrics	[18;35]	{Female, Male}	{Single, Married}	[1;20]
Geometry	[19;29]	{Female, Male}	{Single, Married}	[1;12]
Anatomy	[18;19]	{Female, Male}	{Single}	[1;4]

Single-valued variables

Variable	Description	Type
x_1 = Student #	Unique student identification number	Numerical discrete
x_2 = Course	Student's enrolled course	Categorical
x_3 = Age	Age of student at enrollment date	Numerical discrete
x_4 = Gender	Male, Female	Categorical
x_5 = Marital status	Single, Married, Divorced, ...	Categorical
x_6 = Distance	Distance from campus to home (km)	Numerical continuous



Symbolic variables

Variable	Description	Type
x_2 = Course	Student's enrolled course	Categorical single-valued
x'_3 = Age	Age range of students at enrollment date	Interval-valued
x'_4 = Gender	Set of genders	Categorical multi-valued
x'_5 = Marital status	Set of marital status	Categorical multi-valued
x'_6 = Distance	Distance from campus to home range (km)	Interval-valued

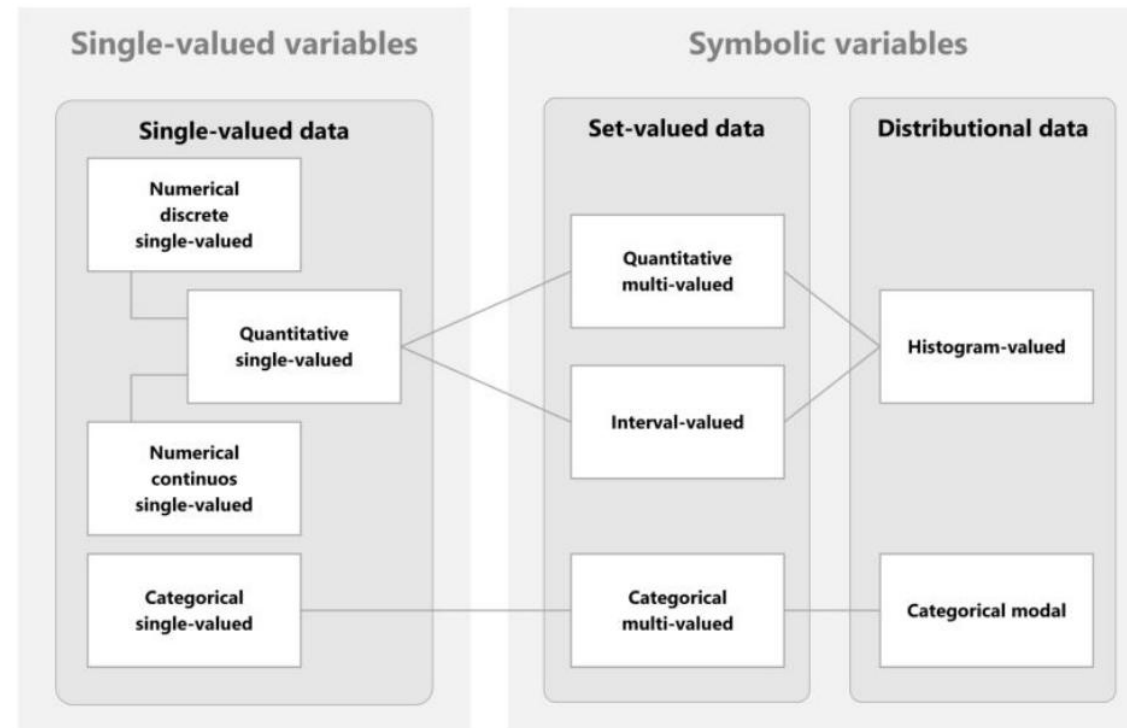
Symbolic Data Analysis to Improve Completeness of Model Combination Methods
Pedro Strecht, João-Mendes Moreira, Carlos Soares, ADMA 2022, Springer, Cham

https://link.springer.com/chapter/10.1007/978-981-99-8391-9_9

Feature engineering

Feature transformation – Discretization (binning)

- Discretization is a form of **Symbolic Data Analysis**



Feature engineering

Case study 1 – Titanic Survival Prediction



- **Preprocessing:** Impute Age, handle missing Embarked; encode categoricals; normalize numeric features
- **Feature engineering:** FamilySize, IsAlone, Title (from Name), ticket group signals; bin Fare/Age if helpful
- **Model:** Logistic Regression (probability-based, discriminative) for interpretability or Decision Tree
- **Evaluation:** Stratified CV; metrics: Accuracy, ROC-AUC

Feature engineering

Case study 2 – House Prices



- **Preprocessing:** Target log-transform; impute domain-aware missing values; one-hot/ordinal encode; fix skewed vars
- **Model:** Gradient Boosting (e.g., XGBoost/LightGBM), optimization-based; strong on tabular with non-linearities
- **Evaluation:** K-fold CV with RMSE on log price; compare to linear baselines (Ridge/Lasso) for interpretability trade-offs

Feature engineering

Case study 3 – Telco Customer Churn



- **Preprocessing:** Clean TotalCharges; encode categoricals; treat imbalance with class weights or SMOTE
- **Model:** Decision Tree (search-based) or C5.0 for explicit, human-readable rules for operations teams
- **Evaluation:** Prioritize Recall/ROC-AUC and cost-aware metrics; calibrate thresholds by retention budget

Feature engineering

Case study 4 – Credit Card Fraud



- **Preprocessing:** Scale Amount/Time; stratified partitions; consider undersampling/oversampling; tune decision threshold
- **Model:** Support Vector Machine (optimization-based, margin) or Gradient Boosting for strong separation
- **Evaluation:** Precision-Recall curves and PR-AUC (recommended); monitor recall at low FPR

Feature engineering

Case study 5 – CIFAR-10 Object Recognition in images



- **Preprocessing:** Normalize pixels; strong data augmentation (flips/crops/color jitter); early stopping
- **Model:** Convolutional Neural Network (optimization-based, parametrized) for spatial hierarchies
- **Evaluation:** Top-1 accuracy; confusion matrix by class; monitor generalization gap